DTU Compute
Department of Applied Mathematics and Computer Science

MakeDo: From README to Test Suite

An Execution Model for Shell Blocks in Markdown

Author: Michele Bandini



MakeDo: From README to Test Suite
An Execution Model for Shell Blocks in Markdown

Master Thesis
July, 2026

By
Michele Bandini

Copyright: Reproduction of this publication in whole or in part must include the customary bibliographic citation, including author attribution, report title, etc.
Cover photo: Vibeke Hempler, 2012
Published by: DTU, Department of Applied Mathematics and Computer Science, Building 324, 2800 Kgs. Lyngby Denmark
https://www.compute.dtu.dk
ISSN: (electronic version)
ISBN: (electronic version)

ISSN: (printed version)
ISBN: (printed version)

Abstract

Over the last two decades, Markdown — a lightweight markup language for formatting plain text — has become ubiquitous in software development. It is consistently used to write READMEs, tutorials, and guides that embed executable shell commands and code snippets. However, as software evolves, this documentation often falls behind, and the embedded code no longer behaves as described — a phenomenon known as documentation drift. We believe the primary cause is that existing testing tools force a trade-off between simplicity and expressiveness: lightweight frameworks can typically only verify stdout, while more expressive tools require separate test artifacts that add complexity and reintroduce the very drift they are meant to prevent. In practice, this trade-off often means validation is skipped entirely, leaving documentation correctness to manual review.

To address this, we introduce MakeDo, a tool that executes shell snippets directly inside Markdown files. It verifies complex system behavior through assertions embedded in the same file as lightweight HTML comments, invisible in the rendered document. With MakeDo, the documentation becomes the test, and the test becomes the documentation.

We evaluate both the prevalence of the problem and the effectiveness of our solution across two popular open-source projects, Hugo and ripgrep. First, a longitudinal study of Hugo's quick-start guide found that its instructions were broken for a cumulative 798 days across a 1,561-commit history, confirming that documentation drift is a severe and persistent issue. Second, applying MakeDo to ripgrep's active user guide surfaced a previously unreported failure. Finally, we show that MakeDo generalizes beyond documentation validation to general-purpose CLI testing by porting Hugo's native integration test suite to Markdown, successfully porting 70 of its 75 tests. Using this ported suite, we evaluate fault detection against ten codebase mutations, achieving detection identical to the native suite, while also surfacing two additional edge cases where full fidelity could not be preserved.

Acknowledgements

I first want to thank my advisor, Christian, for the idea. MakeDo would not exist without him. Thank you for all the insights you provided and for giving me complete freedom during these five months. A big thank you as well to my co-advisor, Ekkart, who always had great advice during our meetings.

I also want to thank all the incredible people I met during my two years in Copenhagen who made this journey so special. Above all, I want to thank Pier, who stood by me through the good times and the ”not so good” ones.

Last but not least, I want to thank my family, who support me from Italy every day: my parents, Alessandra and Riccardo; my sister, Martina; and my two lovely dogs, Donald and George.

Contents

Abstract ii

1 Introduction **1**
1.1 Contributions 2

2 Background and Related Work **4**
2.1 Documentation Decay in Software Projects 4
2.2 Detecting Inconsistency Without Execution 5
2.3 Literate Programming 6
2.4 Literate Testing 6
2.5 Executable Markdown and Shell-Oriented Literate Tools 8

3 MakeDo **11**
3.1 Introduction 11
3.2 Design Goals 11
3.3 Overall Architecture 14
3.4 The MakeDo Document Model 16
3.5 Execution Model 19
3.6 Block Execution Lifecycle 22
3.7 Directive Semantics 25
3.8 Default patterns 28
3.9 Embed Semantics 32
3.10 Implementation 35

4 Evaluation **40**
4.1 Overview 40
4.2 Case Study 1: Prevalence of Drift in Hugo's Quick-Start Guide 40
4.3 Case Study 2: Diagnosing Live Drift in ripgrep's Guide 44
4.4 Case Study 3: General Integration Testing in Hugo 49
4.5 Summary of Findings 53

5 Conclusion **55**
5.1 Summary 55
5.2 Limitations 55
5.3 Future Work 56
5.4 Concluding Remarks 57

6 Declaration of AI Usage **58**
6.1 Development 58
6.2 Writing 58

6.3 Research 59

Bibliography **61**

A Examples of Source-Code Documentation Drift **63**

B Tool Syntax Examples **65**

B.1 `mdsh`: In-Place Output Capture 65

B.2 Runme: Notebook Execution and Shared State 65

B.3 Cram: Transcript Comparison Testing 66

B.4 `clitest`: Inline Markdown Assertions 66

B.5 `byexample`: Dynamic Output Matching 67

C AI Coding Agent Instructions **68**

D Survey of Repository Documentation Validation **71**

E Test Porting Example: `testscript` to MakeDo **74**

1 Introduction

Software projects consist of more than source code. Every codebase has documentation that explains how software should be installed, configured, executed, and used; documents design decisions; and much more. Over the past two decades Markdown [1], a lightweight markup language for formatting plain text, has become the go-to language for this documentation. Nearly every open-source repository contains a README.md and several of them have tutorials and developer guides manually written entirely in Markdown. These documents frequently embed shell commands, configuration snippets, and expected output alongside explanatory text — often a user's very first interaction with a piece of software.

The role of these documents has grown further with the rise of agentic AI. Large Language Models increasingly treat Markdown as an LLM-readable specification, not just human-readable prose: repository documentation, agent instructions, skill files, and tool descriptions are all commonly written in it. This also makes Markdown highly relevant for the future.

Despite its central role, documentation receives none of the scrutiny applied to source code. Teams compile, lint, and test their code before merging. Documentation rarely faces the same scrutiny. Automated tools may check formatting, spelling, or broken links, but developers still check the executable examples inside documentation by hand, if they check them at all.

Unlike code, documentation fails silently: an outdated command triggers no compiler error and no failing test, so it can remain incorrect for years. This is not a rare failure — it is a persistent, measurable problem, known as *documentation drift*. GitHub's 2017 Open Source Survey found that 93% of respondents had encountered incomplete or outdated documentation [2], and a large-scale analysis found that most popular open-source projects had been affected by outdated documentation at some point in their history [3].

To confirm this was still true today, we surveyed 62 popular open-source repositories (see Appendix D) and found that 79% contain executable shell blocks in their primary documentation. However, 67% of these projects perform no CI validation of their Markdown files whatsoever. Furthermore, even among the 33% that do have CI validation, many rely only on tools that check formatting or broken links, rather than executing the embedded shell examples.

Existing tools designed to prevent documentation drift force a trade-off between simplicity and expressiveness. Tools that check *textual* consistency statically cannot catch a shell command that fails because a CLI flag changed or a daemon failed to start. Tools that execute documented commands in-place, such as `mdsh` [4], are self-contained in a Markdown file but their assertion vocabulary is narrow, typically only verifying standard output.

Conversely, tools that support complex assertions, such as Runme [5], often require introducing separate test files (e.g., Bats scripts). This introduces a "two-artefact problem," adding complexity and reintroducing the very drift it was meant to prevent, as the tests and documentation can diverge independently.

This thesis introduces **MakeDo**, a framework that balances both needs by treating Markdown documentation itself as an executable test suite. MakeDo parses a Markdown file, executes its shell blocks, and checks declarative assertions attached to each one. These assertions are written as HTML-comment directives, meaning they do not pollute the rendered document and require no secondary test artefacts. Furthermore, they cover a broad set of observable properties — output, exit status, filesystem state, working directory, and background processes.

MakeDo belongs to the tradition of *literate testing*, exemplified by tools such as Python's `doctest` [6], where documentation and tests become the same artefact. It extends this tradition specifically to command-line software, where correctness depends on more than a function's return value.

This work is guided by three research questions:

- **RQ1 (Prevalence).** How often, and for how long, does executable Markdown documentation diverge from the behaviour of the software it describes?
- **RQ2 (Prevention and Impact).** How can treating documentation as an executable test suite prevent drift in real-world projects, and how intrusive is its integration into existing documentation?
- **RQ3 (Generality).** What are the capabilities and structural limitations of applying a documentation-centric execution model to general command-line integration testing?

We answer these with three case studies: replaying the decade-long commit history of Hugo's `quickstart.md`; analysing ripgrep's `GUIDE.md`, where MakeDo located the exact commit that introduced an existing failure, leading to an issue on the project's GitHub repository[1]; and porting Hugo's own integration test suite to MakeDo, where we evaluated its fault-detection capabilities against a suite of codebase mutations to test its generality beyond documentation.

1.1 Contributions

This thesis makes the following contributions:

- We provide empirical evidence, through a decade-long replay of Hugo's documentation history (Section 4.2), that documentation drift is prevalent and long-lived in real-world executable Markdown.

[1] https://github.com/BurntSushi/ripgrep/issues/3448

- We introduce MakeDo, a directive-based framework for executing Markdown code blocks and asserting on output, exit status, filesystem state, working directory, and background-process behaviour (Chapter 3).
- We show that MakeDo detects and localises real documentation drift, including a previously unknown failure in ripgrep's `GUIDE.md` that we reported upstream (Section 4.3, RQ2).
- We show that MakeDo's execution model generalises to general-purpose CLI integration testing by reproducing Hugo's test suite. We demonstrate equivalent fault detection against ten codebase mutations, while also identifying the structural limitations of pseudo-terminal-based testing by evaluating it against historical stream regressions (Section 4.4, RQ3).

2 Background and Related Work

In this chapter we review the literature relevant to MakeDo and the research questions introduced in Chapter 1. We first examine empirical work on software documentation quality and documentation drift, before surveying approaches to documentation consistency checking, literate programming, executable Markdown systems, and command-line testing frameworks. Together, they establish the context in which MakeDo is positioned and motivate the design decisions evaluated in the following chapters.

2.1 Documentation Decay in Software Projects

That documentation exists is not in question: empirical studies of GitHub repositories consistently find that the vast majority of open-source projects ship a README, and that these files converge on a fairly stable set of content — installation instructions, usage examples, licensing, and contribution guidelines — across languages and ecosystems [7, 8]. That this documentation *stays correct* is a different question, and the evidence here is less reassuring. A broad survey of developers found that *outdated content* is among the most disruptive and frequently cited documentation issues in practice, surpassing concerns about completeness or clarity. These findings echo the two data points already introduced in Chapter 1: GitHub's own developer survey, in which a large majority of respondents had personally encountered incomplete or outdated documentation [2], and a mining study showing that outdated documentation is not a transient condition but one that persists in popular repositories for extended periods once introduced [3]. Taken together, this literature is consistent with RQ1's premise: documentation drift appears to be a recurring, structural feature of how documentation is maintained alongside code, rather than an occasional lapse, and the executable commands embedded in READMEs and guides are exposed to the same neglect as the surrounding prose.

To confirm that this vulnerability persists specifically for executable documentation, we conducted a targeted survey of 62 popular open-source repositories (detailed in Appendix D). We examined their primary documentation (READMEs and quick-start guides) to determine whether they embed executable shell blocks, and whether those blocks are subject to any form of automated continuous integration (CI) validation. We found that while the vast majority of these projects rely on shell blocks to instruct users, most perform no automated validation of those commands whatsoever. Even among projects that do employ documentation linting tools (such as markdownlint or vale), these checks are overwhelmingly restricted to formatting, spell-checking, or dead-link detection. The actual behaviour of the embedded commands is left entirely unverified, confirming that the ”two-artefact” gap remains a systemic vulnerability in modern open-source maintenance.

One natural response to this problem is to stop keeping documentation and code as separate artefacts. By integrating the descriptive text with the source code itself, the hope

is to reduce the friction of keeping them synchronized. As the next section shows, this shift toward embedded documentation has driven research into tools that can check these unified files for internal consistency.

2.2 Detecting Inconsistency Without Execution

One line of research tries to catch documentation drift without running anything, by treating code and its accompanying text as two descriptions that ought to agree and checking them against each other statically. The tools surveyed in this section target narrower artefacts than the READMEs and guides motivating this thesis: source-code comments and docstrings, rather than standalone documentation files. They are included here because they represent the closest existing automated approach to catching documentation drift without execution, and are worth contrasting with MakeDo on method even though they differ in scope. Illustrative examples of the types of documentation drift these tools target are provided in Appendix A.

The earliest work in this vein, iComment, mined natural-language rules out of source-code comments and checked them against the surrounding implementation using program analysis, uncovering real inconsistencies in several large C and Java codebases [9]. @tComment [10] specialised this idea to Javadoc, inferring likely null-handling and exception behaviour from `@param` and `@throws` tags and then generating tests to check whether the implementation actually behaved as documented, combining static extraction with a form of targeted execution. Fraco [11] instead focused on a narrower but common source of drift, comments that refer to identifiers by name, and used part-of-speech tagging and morphological analysis to detect comments left stale after a rename refactoring, outperforming IDE-level rename-and-replace heuristics on the authors' benchmark suite. A later, large-scale mining study of comments and their surrounding methods catalogued the resulting inconsistencies into recurring categories, such as stale parameter descriptions and outdated exception behaviour, and found that comment updates frequently lag behind, or never follow, the code changes that should have prompted them [12].

These tools are effective at what they check, and several combine static analysis with a degree of targeted execution rather than relying on text comparison alone. What they check is nonetheless narrow by construction: a claim expressed in a docstring or comment about a parameter, a return value, or an identifier's name. Few, if any, execute a documented *command* and compare its real-world effects — its exit status, the files it leaves behind, the directory it leaves the user in, or a background process it starts — against a specification; most execute nothing beyond small, targeted test cases synthesised around a single method call. A shell command in a README can be perfectly consistent with the prose around it in exactly the sense these tools check for, and still fail the moment it is run, because the flag it uses was renamed, the file it reads was moved, or the daemon it starts no longer listens on the port the next command assumes. This is the gap identified in Chapter 1: textual-consistency tools catch a different, narrower class of drift than the one MakeDo targets, and the two approaches are complementary rather

than competing.

2.3 Literate Programming

The idea that documentation and code should be maintained as a single artefact is not new. Knuth's concept of literate programming first proposed writing a program as an explanation directed at a human reader, writing narrative text and source code into a single file to ensure they evolved together [13]. The explanation and the implementation live in the same source file, so keeping them consistent is a matter of editing one file rather than two — what this thesis will refer to throughout as the *two-artefact problem*. Later systems relaxed this philosophy into direct interactivity. Computational notebooks — Mathematica and Maple worksheets, and later Jupyter [14], which interleaves Markdown cells with code cells whose text, table, and plot output is captured and stored inline in the same file — let a code fragment and its output live alongside each other in one document, rather than being generated separately as prose and program.

Despite fusing these artefacts in exactly the way literate programming asked for, notebooks still decay in practice. Large-scale studies of notebooks published on GitHub and alongside scientific papers report substantial rot: one study [15] found that a majority of published, in-principle-executable notebooks could not be reproduced by straightforward re-execution, because cells had been run out of the order in which they appear or relied on state left over from edits that were never re-run; another [16], mining over a million notebooks, found a substantial share contained code cells that had never been executed or that could not be re-executed against the notebook's own recorded outputs.

The lesson for this thesis is that interleaving code and prose in a single file is not, by itself, sufficient to prevent drift: the notebook format does not generally compel a document to be re-executed from a clean state or checked against a specification of what should happen, so a cell's stored output can simply be stale from a previous, differently-ordered run without any tooling flagging it. Fusing the artefact appears to solve the synchronisation problem only when something also enforces re-execution and checks the result — which is where literate testing, discussed next, differs from literate programming in general.

2.4 Literate Testing

Literate programming (§2.3) interleaves documentation with executable code, but it does not require that the examples are verified automatically. A narrower idea is *literate testing*, in which the documented examples are themselves executed as tests.

The term *literate testing* is not as well established as *literate programming*, but it is not introduced by this thesis either. The official Python standard-library documentation for `doctest` describes tutorial documentation illustrated with executable input–output examples as having "the flavour of 'literate testing' or 'executable documentation'," depending on whether the examples or the explanatory text are emphasised. Later, it lists "Executable documentation / literate testing" as one of the module's three primary uses [6].

This thesis uses the term in that sense: literate testing is documentation whose examples are executable as tests.

Python's `doctest` module is perhaps the best-known example of this idea. Python allows a string literal to appear as the first statement of a function, class, or module definition. This string, called a *docstring*, is commonly used to describe how the code is intended to be used. `doctest` recognises examples copied from an interactive Python session inside these docstrings and executes them as tests.

Listing 2.1 shows a simple example. The lines beginning with >>> are commands entered into an interactive Python session, often called a REPL (Read–Eval–Print Loop). The lines that follow are the expected results. When `doctest` is run, it re-executes each command and compares the actual output with the documented output. If any result differs, the test fails.

```
def add(a, b):
    """
    Return the sum of two integers.

    >>> add(2, 3)
    5
    >>> add(-1, 4)
    3
    """
    return a + b
```

Listing 2.1: A simple `doctest` example.

`doctest` is effective within its scope and has a long track record as a lightweight way to keep small usage examples honest. The example above also illustrates the limits of that scope: it checks one property, whether the captured standard output of each Python expression matches the expected output exactly. It has little to say about a command's exit status, the state of the filesystem after it runs, the working directory from which a later command should be executed, or a long-running background process a command might start. These concerns matter for command-line software but are outside the scope of evaluating individual Python expressions, which is what `doctest` was designed to check.

The idea illustrated by `doctest` — that documentation is not only written for the reader but can also be executed to verify that the documented behaviour remains true — has since been applied beyond Python docstrings to Markdown documents containing shell blocks. In that setting, the shell block itself becomes the test used to check whether the surrounding documentation is still valid. The tools discussed in the next section take up this idea directly.

2.5 Executable Markdown and Shell-Oriented Literate Tools

A small number of tools extend the idea of literate testing to the shell blocks inside Markdown files, executing them directly rather than comparing text statically. This section surveys five of these tools: `mdsh`, Runme, Cram, `clitest`, and `byexample`. (For concrete examples of the syntax used by each of these tools, please refer to Appendix B.) Each was designed to solve a genuine, and slightly different, problem, and each catches drift that the static tools in §2.2 cannot, because each actually runs a command rather than reasoning about it from its surrounding text. Read together, however, they also show a consistent pattern in what they leave unaddressed, which this section identifies at the end.

`mdsh` is a Markdown shell pre-processor built to keep a README's *example output* synchronised with reality: it scans a file for fenced code blocks introduced by a small comment-based syntax, runs the associated commands, and rewrites the surrounding output block in place, so that re-running `mdsh` updates stale output automatically [4]. Run with its `--frozen` flag in CI, it fails the build if regenerating the file would change it, which is a direct and low-overhead way to catch output drift. Its scope is correspondingly narrow: it checks only whether previously captured text still matches, and, by the maintainers' own admission, it locates blocks with regular expressions rather than a full Markdown parser. It has no vocabulary for exit status, filesystem effects, or background processes, so a command that prints unchanged text while leaving the wrong files behind, or that fails with a non-zero exit code while still printing something plausible, is not distinguished from a correct one.

Runme takes a different approach, aimed less at keeping documentation synchronised and more at making a README directly executable as a notebook. Fenced code blocks can be named and annotated, run individually from a CLI or a VS Code notebook interface, and share state such as environment variables across blocks within one session [5]. This session-sharing model is a genuine advantage over tools that treat each block in isolation. Runme has since added a CI mode in which a project's own Markdown files run as an integration-test suite, but its own documentation is explicit that checking anything beyond a successful run — a particular exit code, an output string, a piece of cluster state — requires an external tool such as Bats, with the command sequence duplicated from the README into a separate `.bats` file. That duplication reintroduces the two-artefact problem that literate programming set out to avoid: the assertions end up living apart from the commands they describe, in a second file that can itself fall out of step with the README it is meant to test.

A third, older approach relies on comparing shell transcripts rather than parsing Markdown. Cram [17], extracted from Mercurial's internal test suite, represents a test as a fenced shell session: lines starting with a two-space-indented `$` run as commands, and the indented lines that follow are compared, literally or against a declared regular expression or glob, to the command's actual output. A mismatch is reported as a diff and can be auto-corrected, which makes Cram well suited to maintaining a large suite of command-

line regression tests. Cram's format is also deliberately close to what a README code block already looks like, and reimplementations such as `craml` carry the same model into other ecosystems. Its assertion vocabulary is, however, restricted to output text and, through a documented exit-code convention, pass, fail, or skip; and because it was built as a general command-line testing tool rather than a documentation tool, it does not operate on Markdown at all. A project cannot point Cram at its README directly; the commands must first be transcribed into Cram's own file format, which recreates the two-artefact problem in a new shape.

`clitest` moves closer to the README itself. Rather than a separate transcript format, it reads a Markdown or plain-text file and treats fenced or indented shell blocks as tests, with expected output declared through inline comments such as `#=>` for output lines and `#=> --exit N` for a specific exit code [18]. This gives `clitest` something the tools above generally lack: a way to state, next to the command itself, what exit status it should produce, without leaving the source file. Its parsing, however, remains a scrape rather than a structural read: `clitest` locates its blocks by matching lines against patterns rather than by walking a Markdown abstract syntax tree, so it can share the same fragility as `mdsh` when a file uses Markdown syntax the matcher does not anticipate. Its assertion vocabulary, while wider than Cram's, still stops at output text and exit code; it offers no way to state that a command should leave a particular file behind, that a directory should be empty afterwards, or that a background process should be listening on a port once a command returns.

`byexample` addresses a different weakness in the tools above. Literal text comparison breaks whenever a command legitimately prints something that varies between runs: a timestamp, a process ID, a version string. `byexample` extracts interactive examples from Markdown and other formats and allows the expected output to contain wildcards — a token such as `<...>` that matches any text — so a line documenting a version command can match `v2.3.1` today and `v2.4.0` after the next release [19]. This handling of dynamic output is a capability none of the other tools surveyed here offer, and is a meaningful gap in their design. At the same time, `byexample` remains, at its core, an input–output matcher in the tradition of `doctest` (§2.4): its concern is whether a command's printed text matches a pattern, not whether the command left the filesystem in a particular state or started a process that is still running once the check completes, so it does not by itself close the gap in filesystem or process assertions left open by the other tools.

Summary. The five tools surveyed in this section differ along a small number of recurring capabilities: whether they read Markdown through a real parser or by matching text patterns; whether they can assert on a command's printed output; whether they can assert on its exit code; whether that output can be matched with a regular expression or wildcard pattern rather than compared literally; whether they can assert on filesystem state, such as a file being created or removed; whether they can assert on a background process, such as one still listening on a port after a command returns; and whether check-

ing non-trivial properties requires introducing a second file alongside the documentation. Table 2.1 summarises where each tool sits on these capabilities.

Table 2.1: Capabilities of existing executable-Markdown and shell-oriented literate testing tools.

Tool	Native Markdown	Output assertions	Exit-code assertions	Pattern matching	File-system	Process assertions	Requires separate test file
`mdsh`	No	Yes	No	No	No	No	No
Runme	Yes	Yes*	Yes*	No	No	No	Yes*
Cram / `craml`	No†	Yes	Yes	Yes	No	No	Yes
`clitest`	No	Yes	Yes	No‡	No	No	No
`byexample`	Partial§	Yes	No¶	Yes	No	No	No

*Runme executes commands in a persistent session and reports successful execution, but richer output and exit-code assertions require integration with an external testing framework such as Bats.

†Cram does not operate directly on Markdown; README examples must first be transcribed into Cram's transcript format.

‡Support for regular-expression or wildcard matching is limited compared with dedicated pattern-matching tools.

§byexample extracts examples from Markdown and other document formats, but does not rely on a full Markdown abstract syntax tree.

¶byexample primarily verifies output rather than providing explicit exit-code assertions.

Taken as a whole, Table 2.1 shows that no single capability is absent from the literature: one tool checks exit codes, another avoids a second file, and yet another matches output with a pattern. What is less common, among the tools reviewed here, is a tool that combines native Markdown parsing with assertions on filesystem and background-process state, while also handling dynamic output and avoiding a second test file. This combination is the gap that motivates the design discussed in the next chapter.

3 MakeDo

3.1 Introduction

Chapter 2 identified a gap in the tools surveyed for executable Markdown: none combines native Markdown parsing with assertions on filesystem and background-process state, while also handling dynamic output and avoiding a second test file (§2.5). This chapter presents MakeDo, a tool built to close that gap.

MakeDo treats Markdown documentation itself as the executable specification. A block may be followed by one or more directives, written as HTML comments beneath it, stating what must hold once the block has run: a string or pattern in its output, an exit code, a working directory, the presence of a file, or the result of a second command. MakeDo walks a file's Markdown abstract syntax tree, runs the blocks sequentially within a single shell session—allowing later blocks to build naturally on the state established by earlier ones—and checks each block's directives against the result. A block with no directives still carries meaning: it asserts, implicitly, that the command exits successfully.

This model is exposed through two commands. `makedo test` runs a file's blocks and reports which directives passed or failed, so a README or guide can be run as a test suite in its own right. `makedo embed` uses the same execution model but writes back into the file, inserting captured output beneath each successful block, so the documentation a reader sees is generated from a real run rather than typed by hand and left to drift. The rest of this chapter sets out MakeDo's design goals, explains the block and directive model a user writes against, describes the execution model that interprets it, and shows how `test` and `embed` are built on that shared machinery.

3.2 Design Goals

Table 2.1 showed that each capability needed to catch command-level drift already exists in some tool, but that none of the tools surveyed combines them. MakeDo's design goals set out to close each of these gaps in one tool, without reopening the two-artefact problem that motivated the shift to embedded documentation in the first place (§2.1).

G1: Execute documentation without translation. Existing executable-documentation tools often require documentation to be translated into a tool-specific representation before it can be verified. Cram requires examples to be rewritten as shell transcripts, while richer assertion frameworks such as Bats move the specification into a separate test file. Even notebook-oriented systems introduce their own execution model and metadata. Although these approaches successfully execute documented commands, they ask authors to adapt their documentation to the tool rather than allowing the tool to execute the documentation as it is written.

MakeDo adopts the opposite philosophy. A Markdown document containing ordinary shell code blocks should already be executable documentation. Verification is introduced through lightweight directives attached directly to those blocks, rather than by rewriting examples into a new language or maintaining a parallel test suite. The documentation therefore remains the primary artefact, with executable behaviour expressed as annotations rather than transformations.

G2: Read Markdown natively. `mdsh` and `clitest` both find shell blocks by matching lines against regular expressions rather than by parsing Markdown structurally (§2.5), which leaves them fragile to syntax the matcher does not expect. MakeDo instead walks a proper Markdown abstract syntax tree, so a block is identified by its structural role in the document — a fenced code block with a recognised shell language identifier — rather than by a pattern over raw text.

G3: Assert on more than captured output. A successful command-line example is characterised by more than its printed output. Every tool surveyed in §2.5 can compare a command's printed output against an expectation, but only some check its exit code, and none checks the state of the filesystem or of a background process once the block returns. A command can print exactly the text a README documents and still fail the reader in every way that matters: it can exit non-zero, leave a stray file behind, or fail to start the daemon the next command assumes is listening. MakeDo therefore adopts a richer assertion model in which correctness is not limited to printed output but can also refer to the execution environment and observable side effects through directives such as `out`, `outr`, `cmd`, `pwd`, and `checkpath` (§3.7).

G4: Preserve the documented execution model. Command-line documentation is typically written as a sequence of commands that build upon one another. Commands frequently change the working directory, export environment variables, initialise projects, or launch long-running services that later commands assume are already running. Requiring documentation to be rewritten into isolated examples changes the semantics of the document itself. MakeDo therefore executes shell blocks sequentially within a single persistent shell session, allowing later blocks to observe the environment established by earlier ones. This mirrors the way readers naturally follow command-line documentation while avoiding artificial execution constraints introduced solely for the benefit of the testing framework.

G5: Require no second artefact. Runme's richer assertions and Cram's transcript format both reintroduce the two-artefact problem in a new shape, moving the specification of correct behaviour out of the README and into a `.bats` file or a Cram transcript that can itself fall out of step with the documentation it tests (§2.5). MakeDo's directives are HTML comments beneath the block they describe, invisible in rendered Markdown by default

but present in the same source file, so the specification and the documentation remain a single artefact.

G6: Support realistic command-line workflows. Installation guides and quick-start tutorials frequently launch long-running processes, such as development servers or database instances, before executing further commands that depend on them. Existing executable-documentation tools generally either wait for such processes to terminate or delegate their management to external testing frameworks. MakeDo instead gives these workflows explicit semantics: once a block's directives have been satisfied, execution proceeds immediately while the process continues running in the background. Active processes are tracked in a registry and are guaranteed to be cleaned up when the document finishes executing (§3.5).

G7: Tolerate legitimate output variation. Literal text comparison breaks on output that legitimately varies between correct executions, such as timestamps, UUIDs, version numbers, or network addresses. Only `byexample` addresses this through unrestricted wildcard matching. MakeDo instead introduces typed placeholders, such as `${{uuid}}`, `${{date}}`, and `${{ip}}`, allowing assertions to constrain the structure of dynamic values while still accepting variation. The same mechanism is used during `embed` to replace runtime values with their placeholder representations, ensuring that generated documentation remains deterministic rather than tied to a single execution (§3.8 and §3.9).

3.3 Overall Architecture

Figure 3.1 presents the overall architecture of MakeDo. The system adopts an interpreter architecture in which a Markdown document is parsed once into an enriched abstract syntax tree (AST) and then interpreted directly. Unlike traditional testing frameworks, which execute handwritten test cases, MakeDo treats the Markdown document itself as the program to be interpreted.

Execution begins with the parser. Besides constructing the ordinary Markdown AST, the parser performs a semantic analysis pass that identifies executable shell blocks. Whenever a fenced code block uses a supported shell language identifier (`bash`, `sh`, `shell`, or `zsh`), the parser examines the immediately following HTML comments. Any comments matching MakeDo's directive syntax are attached to the code block, producing a single `MakeDoBlock`. Directives are optional, so every supported shell block becomes a `MakeDoBlock` regardless of whether directives are present. This transformation is illustrated in Figure 3.2.

Once the MakeDo AST has been constructed, an AST walker traverses the document in source order. Rather than executing shell commands directly, the walker repeatedly passes each `MakeDoBlock` to the block evaluator. The evaluator is responsible for interpreting the semantics of an individual block: executing its shell commands within the shared execution environment, evaluating any attached directives against the observed behaviour, and producing a `BlockOutcome`. This object encapsulates the complete result of evaluating a block, including its execution result, the outcome of each directive evaluation, the overall success or failure of the block, and the processed output used by `embed`.

After a block has been evaluated, control returns to the AST walker, which proceeds to the next `MakeDoBlock`. This cycle repeats until the entire document has been processed. The loop shown in Figure 3.1 reflects this repeated interaction between the walker and the block evaluator.

The `BlockOutcome` abstraction also allows MakeDo's two user commands to share the same execution pipeline. The `makedo test` command consumes each outcome to determine whether the documented assertions succeeded and to generate the final test report. The `makedo embed` command uses the same evaluation results but additionally injects the captured output of successful blocks back into the Markdown document. Consequently, both commands are built on the same interpreter and execution model, differing only in how they consume the resulting `BlockOutcome`.

This separation of responsibilities is deliberate. Parsing is concerned only with constructing a semantic representation of executable documentation; the AST walker controls traversal of the document; the block evaluator interprets the semantics of individual blocks; and the user-facing commands consume the resulting `BlockOutcome`. This modular structure keeps the execution model independent of the parser and allows the following

sections to describe MakeDo's execution semantics without referring to implementation details.

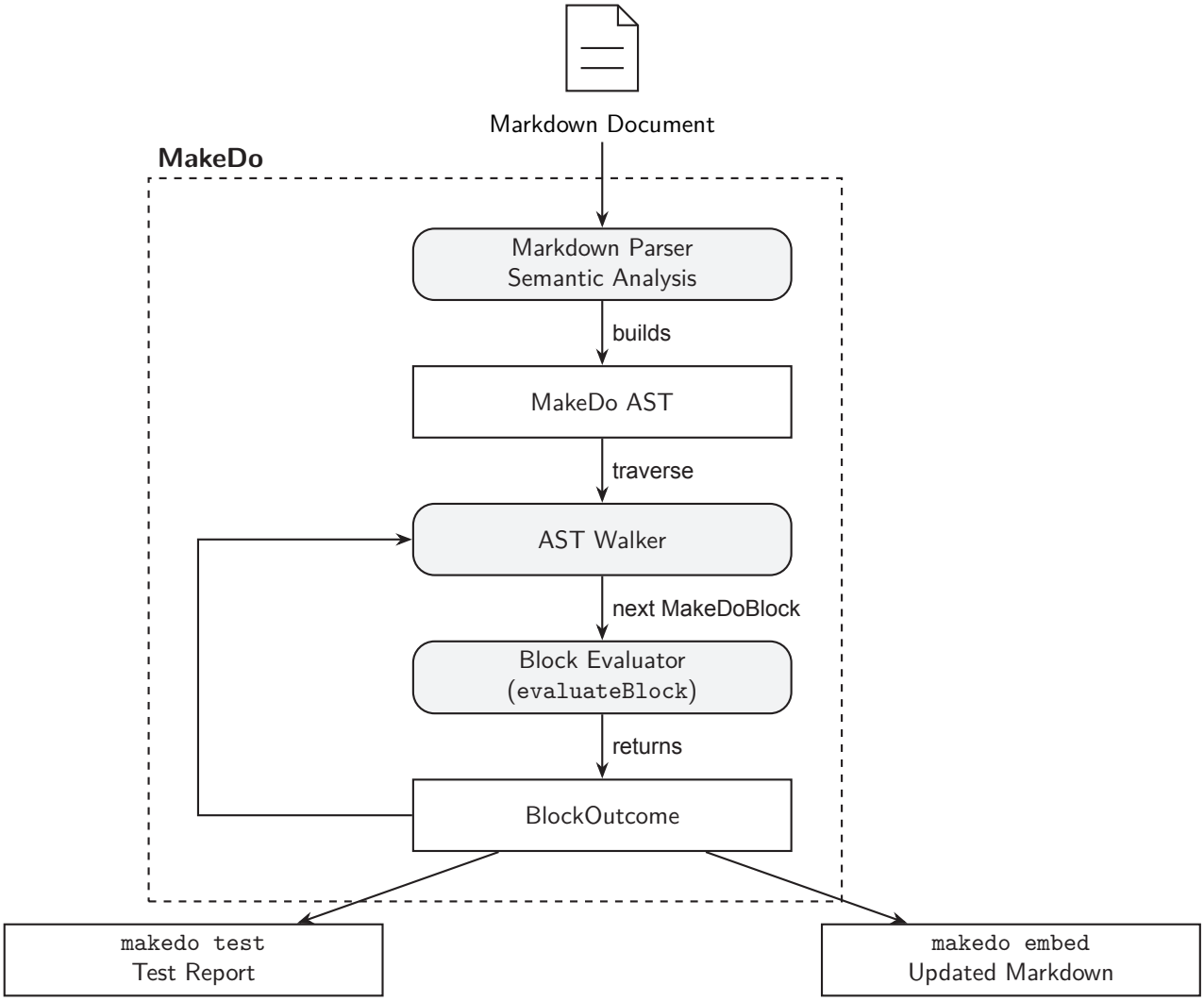


Figure 3.1: Overall architecture of MakeDo. The parser performs semantic analysis while constructing an enriched Markdown abstract syntax tree (AST). The AST walker traverses the document in order and repeatedly passes each `MakeDoBlock` to the block evaluator. The evaluator executes the block, evaluates its directives, and returns a `BlockOutcome` describing the execution result. Control then returns to the AST walker until all executable blocks have been processed. Both `makedo test` and `makedo embed` consume the same `BlockOutcome`, differing only in how they use the evaluation result.

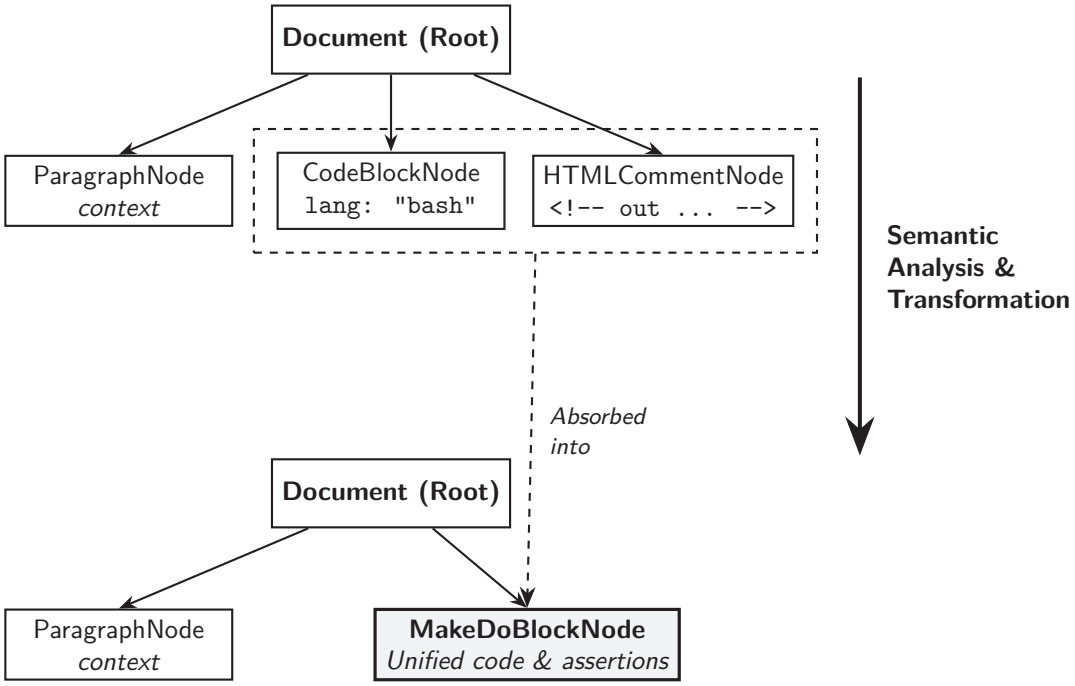


Figure 3.2: AST transformation during parsing. MakeDo's semantic analysis phase identifies a `CodeBlockNode` and any `HTMLCommentNode`s that immediately follow it in the raw AST (top), and absorbs them into a single `MakeDoBlockNode` (bottom), whether or not any directives are present. This yields a unified semantic object linking the executable payload directly with its assertions.

3.4 The MakeDo Document Model

Before describing how MakeDo executes a document, it is useful to define the structure of the artefact it operates on. From MakeDo's perspective, a Markdown document is not simply a sequence of text and formatting elements but a sequence of executable shell blocks extracted from the Markdown abstract syntax tree (AST). As discussed in Section 3.3, the parser performs a semantic transformation that combines each shell code block with any immediately following MakeDo directives into a single semantic object, referred to throughout this chapter as a *MakeDoBlock*. The execution model is defined entirely in terms of these blocks.

We therefore model a MakeDo document as a finite sequence of `MakeDoBlock`s.

Definition 1 (MakeDo document) *A MakeDo document is defined recursively as*

$$m \in \textit{Mdo} ::= b; m \mid \epsilon$$

where each element b is a MakeDo block.

Definition 2 (MakeDo block) *A MakeDo block is defined as*

$$b \in \textit{Blk} ::= \textit{block}(l, c, D)$$

where

$$l \in \textit{Lang} = \{\texttt{bash}, \texttt{sh}, \texttt{zsh}, \texttt{shell}\}$$

$$c \in \textit{Code} = \textit{String}$$

$$D \subseteq \textit{Dir}.$$

The language identifier determines whether a fenced code block is considered executable by MakeDo. The code component c contains the shell program exactly as written in the Markdown document, while D contains every directive parsed from the HTML comments immediately following that code block. Directives are optional, meaning that every supported shell block becomes a MakeDo block, even when $D = \emptyset$.

Listing 3.1 illustrates a simple MakeDo document.

````
Let's create a file.

```bash
echo "Hello world!" > hello.txt
````

Now print the file.

```bash
cat hello.txt
```
<!-- out Hello world -->
````

Listing 3.1: A simple MakeDo document.

The document consists of two MakeDo blocks. The first contains no directives, while the second contains a single output assertion.

$$b_1 = \textsf{block}(\texttt{bash}, \texttt{"echo ``Hello world!'' > hello.txt"}, \emptyset)$$
$$b_2 = \textsf{block}(\texttt{bash}, \texttt{"cat hello.txt"}, d_1)$$

The document itself is therefore represented as

$$[m = b_1; b_2; \epsilon.] \tag{3.1}$$

We now define the directives that form the third component of each MakeDo block.

Definition 3 (Directive) *A directive is defined as*

$$d \in \textit{Dir} ::= \textit{directive}(n, k, s)$$

where

$$\begin{aligned} n &\in \mathbb{B} = \{\mathit{true}, \mathit{false}\} \\ k &\in \textit{Key} = \{\mathit{out}, \mathit{outr}, \mathit{cmd}, \mathit{pwd}, \mathit{checkpath}, \mathit{exit}\} \\ s &\in \textit{String}. \end{aligned}$$

The boolean n represents whether the directive is negated. For example,

```
<!-- out Hello world -->
```

is represented by

$$[d_1 = \text{directive}(\texttt{false}, \texttt{out}, \texttt{"Hello world"}).] \tag{3.2}$$

Similarly,

```
<!-- !out Hello world -->
```

is represented by

$$[\text{directive}(\texttt{true}, \texttt{out}, \texttt{"Hello world"}),] \tag{3.3}$$

indicating that the specified string must *not* occur in the command output.

This document model intentionally captures only the static structure of the executable documentation. It specifies which executable blocks exist and which assertions are attached to them, but it says nothing about the state in which those blocks execute or how their directives are evaluated. Those dynamic aspects are introduced in the next section, where we define MakeDo's execution state.

3.5 Execution Model

The previous section defined the structure of a MakeDo document as a sequence of executable blocks. To define the semantics of execution we must also describe the execution state in which those blocks run.

Unlike many executable-documentation tools, MakeDo does not execute each block in an independent shell. Instead, all blocks within a document are evaluated sequentially inside a single persistent shell session. Consequently, the execution state evolves throughout the execution of the document and is observable by subsequent blocks.

We therefore define the execution state as

$$\sigma \in \text{State} = \text{Shell} \times \text{FS} \times \text{Reg}$$

where

$$\begin{aligned} \text{Shell} &= \text{Env} \times \text{WD} \\ \text{Env} &= \text{Map(String,String)} \\ \text{WD} &= \text{Path} \\ \text{FS} &= \text{Host filesystem} \\ \text{Reg} &= \text{Process registry} \end{aligned}$$

The shell component records the current shell environment. It consists of the currently defined environment variables together with the current working directory. Since MakeDo executes all blocks within the same shell session, modifications performed by one block are visible to every subsequent block.

The filesystem records persistent changes made to files and directories, while the process registry records any background processes that remain active after a block has completed. Together, these three components completely describe the observable execution environment of a MakeDo document.

Having defined the execution state, we can now describe the semantics of executing a single MakeDo block.

A block is evaluated in the current execution state and produces three observable results:

- an updated execution state,
- a verdict indicating whether the block is valid,
- the final output stream produced by the block.

Formally, block execution is modelled as the transition

$$[\![b]\!] : \text{State} \to \text{State} \times \mathbb{B} \times \text{Output}$$

or equivalently,

$$[\![b]\!](\sigma) = (\sigma', v, o_f)$$

where

$$\begin{aligned} \sigma, \sigma' &\in \text{State} \\ v &\in \mathbb{B} = \{\text{true}, \text{false}\} \\ o_f &\in \text{Output} \end{aligned}$$

The updated state σ' captures any observable effects produced by executing the block. Since MakeDo executes an entire document within a persistent shell session, these effects include modifications to the filesystem, the process registry, and the shell environment that later blocks may observe.

The verdict v represents the semantic result of the block. It is `true` only if the execution satisfies the behaviour specified by its directives. If the block contains no directives, v is determined solely by whether the shell command exits successfully.

The value o_f denotes the final output stream produced during execution. It consists of the multiplexed contents of standard output and standard error and is preserved independently of the verdict. Although primarily used later by the `embed` command, it forms part of the semantic result of every block because it is an observable effect of execution.

Since a document is defined as a sequence of blocks, document execution is obtained by repeatedly applying the block semantics.

Given

$$m = b_1; b_2; \cdots; b_n; \epsilon$$

execution proceeds sequentially:

$$
\begin{aligned}
(\sigma_1, v_1, o_1) &= [\![b_1]\!](\sigma_0) \\
(\sigma_2, v_2, o_2) &= [\![b_2]\!](\sigma_1) \\
&\vdots \\
(\sigma_n, v_n, o_n) &= [\![b_n]\!](\sigma_{n-1})
\end{aligned}
$$

where the output state of one block becomes the input state of the next.

This definition captures the fundamental semantics of MakeDo: document execution is a deterministic traversal of a sequence of executable blocks, each transforming the execution state while producing both a validation verdict and an observable output.

The remaining question is how the verdict v is determined. Unlike conventional shell scripts, not every block reaches its verdict in the same way. Some commands terminate naturally before their directives are evaluated, whereas others intentionally continue running while later blocks execute. MakeDo therefore distinguishes between foreground and background execution, which is described in the following section.

````
Start a local HTTP server

```bash
python3 -m http.server 8000 &
```
<!-- out Serving HTTP -->

Verify that the server is responding

```bash
curl -s localhost:8000
```
<!-- out Directory listing -->
````

Listing 3.2: A document demonstrating a background process.

3.6 Block Execution Lifecycle

The previous section defined the execution state and established that a MakeDo document executes as a sequence of blocks within a single persistent shell session. We can now complete the execution model by describing how an individual block is evaluated.

At first sight, block execution appears straightforward: execute the shell command, wait for it to finish, and then evaluate the directives. This model is sufficient for short-lived commands such as `mkdir`, `cp`, or `cat`, all of which terminate naturally after producing their observable behaviour. However, command line documentation frequently contains commands that intentionally do not terminate. Installation guides often start a database server, development server, or message broker before demonstrating how to interact with it. Waiting for these commands to exit would prevent the remainder of the document from executing.

Consider the following example.

The first block launches a web server that is expected to remain running throughout the execution of the document. The second block immediately depends on that server being available. If MakeDo simply waited for the first command to terminate before continuing, the second block would never execute. Conversely, immediately continuing without first checking that the server had started correctly would risk executing the second block against a server that was still starting or had already failed.

MakeDo therefore adopts a different execution strategy. Directives are evaluated continuously while a block executes rather than only after the underlying shell command exits. As soon as every directive attached to a block has been satisfied, MakeDo has obtained sufficient evidence that the documented behaviour is correct and may safely continue executing the remainder of the document.

Execution nevertheless remains sequential. Every ordinary block executes inside the same persistent shell session introduced in the previous section, inheriting the current working directory and shell environment established by all preceding blocks. Only com-

mands intentionally placed in the background continue executing concurrently after their directives have been satisfied. These processes are recorded in the process registry, allowing later blocks to interact with them while guaranteeing that they are terminated when document execution finishes.

Figure 3.3 illustrates this execution model. The main execution flow proceeds sequentially through the persistent shell, while background processes continue independently after the evaluator has determined that they satisfy the required directives.

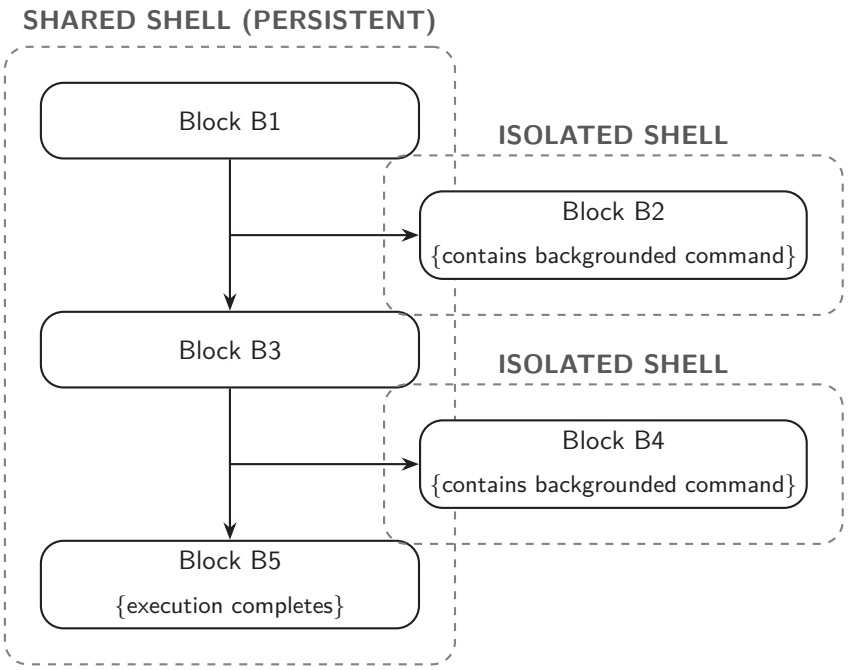


Figure 3.3: Architectural diagram illustrating main-thread execution within a Shared Shell. Background processes fork directly from the execution flow into distinct Isolated Shells.

From the perspective of the evaluator, every block therefore reaches one of two completion modes.

$$\delta \in \text{CompletionMode} = \{\text{Completed}, \text{Ready}\} \tag{3.4}$$

Completed. A block reaches the `Completed` mode when its foreground execution terminates before MakeDo has enough information to conclude its evaluation. Once the process exits, the complete output stream, final shell state, and filesystem state are available for a final evaluation of the attached directives. This is the normal behaviour for short-lived commands.

Ready. A block reaches the `Ready` mode when every attached directive has already been satisfied while the command is still executing. Rather than waiting for the underlying process to terminate, MakeDo concludes that the documented behaviour has been successfully demonstrated and continues executing the next block immediately. Any pro-

cesses that remain alive are recorded in the process registry and are terminated automatically once execution of the document has completed.

The execution semantics introduced in the previous section can therefore be refined as

$$[\![b]\!](\sigma) = (\sigma', \delta, v, o_f) \tag{3.5}$$

where

$$\begin{aligned} \sigma' &\in \text{State} \\ \delta &\in \text{CompletionMode} \\ v &\in \mathbb{B} \\ o_f &\in \text{Output} \end{aligned}$$

The updated state σ' captures the observable changes made by the block, including modifications to the shell environment, filesystem, and process registry. The completion mode δ records how MakeDo concluded evaluation of the block, whereas the verdict v records whether the block satisfied its directives. Finally, o_f denotes the multiplexed output stream produced during execution, which forms part of the semantic result of the block and is later reused by the `embed` command.

3.7 Directive Semantics

The previous sections defined what constitutes a MakeDo document and described how blocks are executed. Execution alone, however, is not sufficient to determine whether a block is correct. A command may finish successfully while producing the wrong output, leaving the filesystem in an unexpected state, or failing to establish the environment expected by subsequent blocks. The purpose of directives is therefore to express the observable properties that must hold before a block is considered successful.

As introduced in Section 3.4, a directive is represented by the tuple

$$
\begin{aligned}
&d \in \text{Dir} ::= \text{directive}(n, k, s)\\
&n \in \mathbb{B}\\
&k \in \text{Key}\\
&s \in \text{String}
\end{aligned}
$$

where n denotes whether the directive is negated, k identifies the directive type, and s stores the directive argument.

The meaning of a directive is defined by an evaluation function

$$[\![d]\!] : (O, \sigma) \to \mathbb{B},$$

which evaluates a directive against the observed execution output O and the current execution state σ. The result is a boolean value indicating whether the directive holds.

A block may contain zero or more directives. Their semantics are conjunctive: every directive attached to a block must evaluate to `true`.

$$[\![D]\!](O, \sigma) = \bigwedge_{d \in D} [\![d]\!](O, \sigma).$$

Consequently, the verdict introduced in the previous section is defined as

$$v = [\![D]\!](O, \sigma).$$

The semantics of each directive are now defined individually.

3.7.1 `out`

The `out` directive succeeds whenever its argument appears as a substring of the multiplexed output stream.

$$[\![\texttt{out}(s)]\!](O, \sigma) = (s \subseteq O).$$

3.7.2 `outr`

The `outr` directive interprets its argument as a regular expression and succeeds if the output stream matches it.

$$[\![\texttt{outr}(r)]\!](O, \sigma) = \mathsf{regexMatch}(r, O).$$

3.7.3 `pwd`

The `pwd` directive asserts that the shell's current working directory matches the supplied path.

$$[\![\texttt{pwd}(p)]\!](O, \sigma) = \mathsf{cwd}(\sigma) = p.$$

3.7.4 `checkpath`

The `checkpath` directive succeeds if the specified file or directory exists within the filesystem component of the execution state.

$$[\![\texttt{checkpath}(x)]\!](O, \sigma) = x \in \mathsf{FS}(\sigma).$$

3.7.5 `cmd`

Unlike the previous directives, `cmd` does not inspect the output of the current block. Instead, it executes a second command within the current shell environment and succeeds only if that command terminates with exit status zero.

$$[\![\texttt{cmd}(c)]\!](O, \sigma) = \mathsf{exitCode}(\mathsf{exec}(c, \sigma)) = 0.$$

3.7.6 `exit`

The `exit` directive succeeds if the executed command's termination status matches the specified exit code.

$$[\![\texttt{exit}(e)]\!](O, \sigma) = (\mathsf{exitCode}(\sigma) = e).$$

3.7.7 Negation

Finally, every directive admits a negated form by prefixing its keyword with the `!` symbol. Negation simply inverts the result of the underlying predicate.

$$[\![!d]\!] = \neg[\![d]\!].$$

For example, the directive

```
<!-- !out Hello World -->
```

evaluates to `true` precisely when the string `Hello World` does *not* occur within the output stream.

The directive model deliberately remains small. Each directive observes one aspect of the execution state or output, while conjunction allows them to be combined into richer specifications. For example, a block may simultaneously assert that a server has printed its startup message, that its working directory is correct, and that a configuration file has been created before execution is allowed to continue. This compositional approach keeps the directive language compact while allowing expressive behavioural specifications to be embedded directly alongside the documentation they verify.

````
5. Verify that you've installed Go by opening a command prompt and typing
the following command:

```bash
go version
```
Confirm that the command prints the installed version of Go.
````

Listing 3.3: A typical command-line verification step from the Go documentation.

````
5. Verify that you've installed Go by opening a command prompt and typing
the following command:

```bash
go version
```
<!-- out go1.24.7 -->
Confirm that the command prints the installed version of Go.
````

Listing 3.4: A brittle test asserting an exact Go version.

3.8 Default patterns

A fundamental challenge when combining executable testing with static documentation is data volatility. Commands frequently produce dynamic runtime data—such as UUIDs, timestamps, or ephemeral IP addresses—whereas documentation generally requires deterministic, static text to remain readable and to avoid perpetual version control diffs.

To bridge this gap, MakeDo introduces *Dynamic Typing*, a formal mechanism that allows authors to assert the structural shape of an output rather than a specific, transient instance.

3.8.1 The Volatility Problem

Consider a real-world scenario from the official Go installation documentation[1], which instructs users to verify their installation by checking the command line version.

If a user has installed version 1.24.7, executing this block yields:

```
go version go1.24.7 linux/amd64
```

To formalize this step into a MakeDo test, a naive approach would assert the exact output using the `out` directive:

This approach is inherently brittle: the test fails as soon as the environment's Go version is updated. While MakeDo offers the `outr` directive to perform regular expression matching (e.g., `<!-- outr go\d+\.\d+\.\d+ -->`), requiring document authors to write complex regular expressions for standard data types degrades the authoring experience and introduces syntactic noise.

[1] https://go.dev/doc/install

````
5. Verify that you've installed Go by opening a command prompt and typing
the following command:

```bash
go version
```
<!-- out go${{version}} -->
Confirm that the command prints the installed version of Go.
````

Listing 3.5: A robust test using a dynamic type placeholder.

3.8.2 Formalizing Placeholders and Automatic Promotion

To resolve this, MakeDo provides a set of pre-configured structural placeholders. At the time of writing, the environment defines a type registry Γ_{type}, which maps a placeholder string to its underlying regular expression:

Type Placeholder	Underlying Regular Expression Pattern
`date`	`\d{4}-\d{2}-\d{2}`
`time`	`\d{2}:\d{2}:\d{2}`
`uuid`	`[0-9a-fA-F]{8}-[0-9a-fA-F]{4}-[0-9a-fA-F]{4}-[0-9a-fA-F]{4}-[0-9a-fA-F]{12}`
`ip`	`\d{1,3}(?:\.\d{1,3}){3}`
`number`	`[-+]?\d*\.?\d+`
`version`	`\d+\.\d+(?:\.\d+)?(?:\.\d+)?`

Table 3.1: The Γ_{type} registry mapping MakeDo types to their regex definitions.

Authors invoke these types using the syntax `${{type}}`. Consequently, the brittle assertion from Listing 3.4 is elegantly refactored:

Note that the placeholder is used inside of an `out` directive rather than an `outr` one. This provides syntactic sugar over `outr`, keeping the semantics of `out` clean and hiding regular expressions from the author, while internally compiling to regex to perform the match.

3.8.3 Directive transformation

We now formalize the transformation function Φ introduced above, treating it as a compositional translation from the syntax of `out` literals into the semantic domain of regular expressions.

We now make Φ precise by walking through its definition alongside the running example from Listing 3.5, where the directive body is

$$S = \texttt{go\$\{\{version\}\}}.$$

Decomposing the directive. The first observation is that any directive body is just an alternation of *literal text* and *placeholder occurrences*. For our example, S splits into a

literal span `"go"`, followed by a single placeholder `${{version}}`, followed by an empty literal span. In general we write this decomposition as

$$S \;=\; \ell_0\,\texttt{\$\{\{}\tau_1\texttt{\}\}}\,\ell_1\,\texttt{\$\{\{}\tau_2\texttt{\}\}}\cdots\texttt{\$\{\{}\tau_n\texttt{\}\}}\,\ell_n, \qquad \ell_i \in \Sigma^*,\ \tau_i \in \mathsf{TypeId},$$

which for our example instantiates to $n = 1$, $\ell_0 = \texttt{"go"}$, $\tau_1 = \texttt{version}$, and $\ell_1 = \texttt{""}$ (the empty string, since nothing follows the placeholder).

Escaping the literal spans. Literal spans must be turned into regular expressions that match *exactly* that text, character for character. This is the job of the quote-meta function $\mathsf{qm} : \Sigma^* \to \mathcal{R}$, which escapes any regex metacharacters in its input. In our example $\ell_0 = \texttt{"go"}$ contains no metacharacters, so

$$\mathsf{qm}(\texttt{"go"}) \;=\; \texttt{go}.$$

(Had the literal instead been, say, `"v1.2"`, qm would have escaped the period: $\mathsf{qm}(\texttt{"v1.2"}) = \texttt{v1\.2}$, so it matches a literal dot rather than "any character.")

Resolving the placeholders. Each placeholder τ_i is looked up in the type registry $\Gamma_{\mathsf{type}} : \mathsf{TypeId} \rightharpoonup \mathcal{R}$ from Table 3.1. For our example, $\tau_1 = \texttt{version}$, and consulting the table gives

$$\Gamma_{\mathsf{type}}(\texttt{version}) \;=\; \texttt{\textbackslash d+\textbackslash .\textbackslash d+(?:\textbackslash .\textbackslash d+)?(?:\textbackslash .\textbackslash d+)?}.$$

If a placeholder's type identifier is *not* in $\mathsf{dom}(\Gamma_{\mathsf{type}})$ — e.g. a typo like `${{version}}` — we do not want compilation to fail outright; instead Φ falls back to treating it as literal text via $\mathsf{qm}(\texttt{\$\{\{}\tau_i\texttt{\}\}})$, so a misspelled type simply fails to match at runtime rather than breaking the build.

Putting it together. Φ concatenates these pieces in order: the escaped literal span, then the resolved placeholder, then the next escaped literal span, and so on:

$$\Phi(S) \;=\; \mathsf{qm}(\ell_0) \;\cdot\; \prod_{i=1}^{n} \Gamma_{\mathsf{type}}(\tau_i) \cdot \mathsf{qm}(\ell_i).$$

Instantiating this for our running example ($n = 1$, $\ell_0 = \texttt{"go"}$, $\tau_1 = \texttt{version}$, $\ell_1 = \texttt{""}$) gives

$$\Phi(\texttt{go\$\{\{version\}\}}) \;=\; \texttt{go}\cdot\texttt{\textbackslash d+\textbackslash .\textbackslash d+(?:\textbackslash .\textbackslash d+)?(?:\textbackslash .\textbackslash d+)?} \;=\; \texttt{go\textbackslash d+\textbackslash .\textbackslash d+(?:\textbackslash .\textbackslash d+)?(?:\textbackslash .\textbackslash d+)?},$$

which is exactly the compiled pattern that `outr` evaluates against the execution buffer, and which correctly matches the observed output `go1.24.7` regardless of which patch version is installed.

The degenerate case. Finally, note what happens when S has no placeholders at all (i.e. $n = 0$, so $S = \ell_0$): the product is empty and $\Phi(S) = \mathtt{qm}(\ell_0) = \mathtt{qm}(S)$. This means a plain `out` directive like `<!-- out Done -->` is simply matched against itself exactly — Φ is a conservative generalization of the original `out` semantics, not a replacement for it.

3.9 Embed Semantics

The previous sections defined how MakeDo executes a document and how it judges each block against its directives. `makedo test` stops there: it reports a verdict and discards the rest. `makedo embed` does not stop there. It takes the same execution and writes the command's output back into the document, so the reader sees real output rather than a verdict alone. This section defines that step.

3.9.1 Extending the block model

A plain `MakeDoBlock` carries a language, a command, and a set of directives (§3.4). Embed needs one more piece: the output the block produced when MakeDo last ran it. We extend the block definition accordingly.

Definition 4 (Embedded block) *An embedded block is defined as*

$$b^E \in \textit{Blk}^E ::= \textit{block}^E(l, c, D, o)$$

where l, c, and D are as in Definition 1, and

$$o \in \textit{Output} \cup \{\bot\}.$$

Here o is the block's captured output, and $\bot$ marks a block with no output to show — either because embed has not yet run, or because the block failed and has nothing worth writing back.

3.9.2 From execution to embedded document

Section 3.5 defined block execution as a transition $[\![b]\!](\sigma) = (\sigma', \delta, v, o_f)$ and showed how a document runs as a sequence of such transitions. Embed reuses this sequence directly; it adds nothing to execution itself. It only decides, for each block, what to keep.

Definition 5 (Embed) *Given a document $m = b_1; b_2; \cdots; b_n; \epsilon$ and its execution trace $(\sigma_i, \delta_i, v_i, o_i)$ for $i = 1, \ldots, n$ (§3.5), embed is the function*

$$\text{Embed}(m) = b_1^E; b_2^E; \cdots; b_n^E; \epsilon$$

where

$$b_i^E = \textit{block}^E(l_i, c_i, D_i, o_i^E), \qquad o_i^E = \begin{cases} o_i & \textit{if } v_i = \textit{true} \\ \emptyset & \textit{if } v_i = \textit{false.} \end{cases}$$

A failed block keeps its code and its directives untouched but gains no output. This matters: embed must never present the reader with output from a command that did not do what the document claims it does. Only a block that satisfied its directives earns a place in the rewritten file.

3.9.3 Rendering

Embed(m) is still an AST, not a file. Writing it back to Markdown means rendering each b_i^E in source order. A block with $o = \emptyset$ renders exactly as it did before: the fenced code block followed by its directive comments, unchanged. A block with $o \neq \bot$ renders the same way, but with one addition: a fenced code block holding o, inserted directly beneath the directive comments.

The directives remain HTML comments, invisible in rendered Markdown (G5, §3.2). The inserted output does not; it is ordinary Markdown, meant to be read. This is the difference between the two commands in one sentence: `test` reads the directives and throws away the output, and `embed` reads the directives and keeps the output instead of the verdict.

3.9.4 Keeping embedded output deterministic

Running the same document twice can produce two different outputs — a new UUID, a later timestamp — even when both runs are equally correct. Embedding the raw value from a single run would make the generated document just as brittle as we saw in §3.8, and would leave every later run producing a difference.

Embed avoids this by normalising output through the same type registry Γ_{type} used by Φ. Where a block's directive already names a placeholder, embed replaces the matching span of the captured output with that placeholder before insertion, rather than with the literal value.

Definition 6 (Output normalisation) *Given output o and a directive $d = \text{directive}(n, \texttt{out}, s) \in D$ whose body s contains a placeholder τ, let M be the span of o matched by $\Gamma_{\mathsf{type}}(\tau)$ under $\Phi(s)$ (§3.8). Normalisation replaces M in o with the literal text $\$\{\{\tau\}\}$, giving o_i^E in place of o_i.*

A block with no placeholder directive is left alone: normalisation only ever touches output the author has already marked as variable.

3.9.5 Example

Recall Listing 3.1. Running it produces $v_1 = v_2 = \texttt{true}$, with o_1 empty (`echo` writes only to the file) and $o_2 =$ "`Helloworld!`". Embed writes both outcomes back into the file:

The first block gains no output block, since o_1 is empty and adds nothing for a reader. The second gains one, placed beneath its directive and holding the exact text the command printed. Nothing here is invented: every line embed writes came from a real run of the document, and every line disappears again the moment the block that produced it stops passing.

Now take Listing 3.5, whose directive names the `version` placeholder. A run on a machine with Go 1.24.7 installed produces $o =$ "`go1.24.7 linux/amd64`". Normalisation finds the span `1.24.7` matched by $\Gamma_{\mathsf{type}}(\texttt{version})$ under Φ, and replaces it before embedding:

A later run on a different machine, with a different Go version installed, produces the identical file. The directive already told embed which part of the output was never meant

````
Let's create a file.

```bash
echo "Hello world!" > hello.txt
```
Now print the file.



```bash
cat hello.txt
```
```stdout
Hello world!
```
````

Listing 3.6: The document from Listing 3.1 after `makedo embed`.

````
```bash
go version
```
<!-- out go${{version}} -->
```
go${{version}} linux/amd64
```
````

Listing 3.7: Embedded output with a typed placeholder preserved.

to be fixed; embed simply takes it at its word.

3.10 Implementation

The preceding sections defined MakeDo's model independently of any particular implementation. MakeDo is implemented in Go[2]. This section describes how the model's central objects — the Markdown AST, the persistent shell, and the process registry — are realised in practice, and notes where the implementation currently diverges from the formal model.

3.10.1 Command-line interface

MakeDo is exposed as a single binary built around Cobra[3], a widely used Go library for structuring command-line applications as a tree of subcommands. `test` and `embed` (§3.1) are implemented as subcommands of the root command that share a common execution path: both parse the target file into a MakeDo AST, run the same interpreter over it, and only diverge once a `BlockOutcome` has been produced, mirroring the shared-pipeline architecture of Figure 3.1.

3.10.2 Parsing and directive extraction

Markdown parsing is delegated to Goldmark[20], a CommonMark-compliant[1] Go library that exposes the document as a navigable AST rather than a flat token stream. This is what allows MakeDo to satisfy G2 (§3.2): a shell block is identified by walking Goldmark's `FencedCodeBlock` nodes and checking their language attribute against Lang $=$ $\{\texttt{bash}, \texttt{sh}, \texttt{zsh}, \texttt{shell}\}$, rather than by matching patterns over raw text.

Delegating to Goldmark also means MakeDo inherits CommonMark's fence syntax for free, rather than committing to one specific fence style. CommonMark permits a code fence to be opened with three or more backticks or three or more tildes (`~~~`), with the closing fence required only to be at least as long as the opening one. A tool that recognised shell blocks by matching literal ```` ```bash ```` at the start of a line — as `mdsh` and `clitest` do (§2.5) — would silently skip a block fenced with four backticks or with tildes, even though both are valid Markdown and semantically identical to the three-backtick case. Because MakeDo instead asks Goldmark's AST for each fenced block's language and content directly, the specific fence syntax an author happened to use is irrelevant to whether MakeDo recognises the block; this is a direct consequence of G2 rather than a feature implemented on top of it.

Command-transcript and split-block formats. Command-line documentation is not always written as a bare shell block. A common convention, especially in READMEs aimed at first-time users, prefixes each command with a `$` prompt and shows its output on the following lines within the *same* fenced block, as demonstrated in Listing 3.8.

This is semantically the same document as writing the command in its own block and its expected output as a directive or a following block, but it is a different concrete syntax, and a parser that expects one cannot read the other without the author rewriting their

[2] https://github.com/MarlonBando/makedo
[3] https://github.com/spf13/cobra

````
```bash
$ echo "Hello world"
Hello world
```
````

Listing 3.8: An example of a command-transcript format showing both the input prompt and output within a single fenced block.

documentation to suit the tool — precisely the friction G1 rules out. MakeDo therefore treats the two as equivalent inputs. When a fenced shell block's content begins with a `$`-prefixed line, MakeDo treats every consecutive `$`-prefixed line as executable code and every line beneath it as that command's output. This is parsed into the same `MakeDoBlock` representation as an ordinary block would be regardless of which convention the author used, so an author already writing dollar-prompt transcripts does not need to translate their documentation into MakeDo's preferred style before it becomes executable.

The semantic analysis pass illustrated in Figure 3.2 is implemented as a single traversal following parsing: for each qualifying fenced code block, MakeDo inspects the sibling nodes immediately below it and collects any `HTMLBlock` nodes whose content matches the directive comment syntax `<!-- keyword content -->`. These are parsed into `directive`(n, k, s) tuples (§3.4) and attached to the enclosing block, producing the `MakeDoBlock` that the rest of the pipeline operates on. Because directives are ordinary HTML comments, they are invisible when the file is rendered as Markdown by default, and only appear if the author explicitly wraps them in backticks — the mechanism underlying G5.

3.10.3 The persistent shell session

Section 3.5 defines Shell $=$ Env $\times$ WD and requires that all blocks in a document observe the cumulative effect of every block that ran before them. A naive implementation — spawning a fresh subprocess per block and passing state explicitly — cannot reproduce this faithfully: it cannot, for instance, transfer a `cd` or an exported environment variable to the next block without MakeDo reconstructing shell state itself.

MakeDo instead runs the entire document inside a single pseudo-terminal (pty). At the start of execution, MakeDo forks a shell process attached to a pty pair and keeps it alive for the lifetime of the run. Because the shell process itself is never restarted, Env and WD persist naturally between blocks exactly as Section 3.5 requires — MakeDo does not need to track or replay environment changes itself, since the underlying shell already does. A pty also matters for what MakeDo captures: many commands documented in READMEs — package managers, compilers, interactive installers — detect whether their standard output is a terminal and change their behaviour accordingly, for instance by disabling colour codes or progress output when writing to a pipe. A pty presents MakeDo's shell with what looks like a real terminal, so the output MakeDo captures, and later embeds (§3.9), matches what a reader running the same command would actually see.

A shared shell still leaves one problem open: how does MakeDo know when a block has

```
git commit -m "Update docs"
git push origin main
```

Listing 3.9: A standard Git workflow executed as a MakeDo block.

```
git commit -m "Update docs"
git push origin main
EXIT_CODE=$?
pwd > /tmp/makedo_cwd_1716120000
export -p > /tmp/makedo_env_1716120000
echo ""
echo "MAKEDO_DONE_1716120000: $EXIT_CODE"
```

Listing 3.10: The synthesized wrapper script MakeDo writes to the pseudo-terminal.

finished, without imposing structure of its own on the command the author wrote? MakeDo answers this by wrapping each block's code in a short script before writing it to the pty, rather than sending the code verbatim. Given a block's code c, MakeDo constructs and executes:

1. A unique identifier, derived from the current Unix timestamp, generated once per block to avoid collisions with markers from earlier or later blocks.
2. The block's code c, executed exactly as written.
3. An immediate capture of the exit status, `EXIT_CODE=$?`, run directly after c so that no intervening command can overwrite it.
4. A dump of the resulting shell state to two temporary files, `pwd > /tmp/makedo_cwd_<UUID>` and `export -p > /tmp/makedo_env_<UUID>`, recording the working directory and exported environment exactly as c left them.
5. An empty `echo ""`, guarding against a block whose final output has no trailing newline — without it, MakeDo's marker could be appended directly to the block's own output and go undetected, stalling the run.
6. A marker line, `echo "MAKEDO_DONE_<UUID>: $EXIT_CODE"`, written last.

To illustrate this mechanism, consider a block containing a standard Git workflow, as shown in Listing 3.9.

Instead of sending this directly to the shell, MakeDo generates a timestamp (e.g., `1716120000`) and synthesizes the script shown in Listing 3.10, which it writes directly to the pty.

As the shell executes the script, MakeDo reads the pty's multiplexed `stdout`/`stderr` stream continuously, buffering the output and watching for the marker. The empty `echo ""` ensures the marker always begins on a new line. The resulting stream looks like the output detailed in Listing 3.11.

Once the marker line appears, MakeDo stops consuming output for the current block. It

```
[main a1b2c3d] Update docs
 1 file changed, 2 insertions(+)
To https://github.com/user/repo.git
   d9f8e7a..a1b2c3d  main -> main

MAKEDO_DONE_1716120000: 0
```

Listing 3.11: The multiplexed output stream buffered by MakeDo.

parses the exit code (`0`) from the marker line, reads the two state files to recover Env and WD, and discards the temporary files. This is the point at which a foreground block reaches $\delta = \text{Completed}$ (§3.6): the full output, exit code, and resulting state are all available, and the block's directives are evaluated against them before the walker moves on.

3.10.4 Background processes and the process registry

The marker-based scheme above assumes a block eventually finishes. Section 3.6 identifies exactly the case where this assumption fails: a block that starts a long-running process, such as the `python3 -m http.server` example, is never expected to print a completion marker at all. MakeDo detects this case lexically, by the presence of a trailing background operator (`&`) in the block's code, and handles it differently from an ordinary foreground block.

Rather than writing the command into the shared pty, MakeDo spawns a second, independent pty for it. Before doing so, MakeDo sources the working directory and environment already dumped from the shared session's temporary files (§3.10.3) into this new shell, so the background command starts with the same Env and WD the shared session had reached at that point. This is the isolated shell shown in Figure 3.3: the background command runs independently of the shared session from this point on, leaving the shared pty free to continue executing the next block in sequence.

Because this process may never terminate, MakeDo cannot wait for a marker before evaluating its directives. Instead it reads the isolated pty's output incrementally and re-evaluates the block's directive set D against the output accumulated so far after every read. As soon as $[\![D]\!](O, \sigma)$ holds, the block reaches $\delta = \text{Ready}$: MakeDo records the process's ID in the process registry Reg and the walker (§3.3) proceeds immediately to the next block in the shared session, without waiting for the backgrounded process to exit. Later blocks can still interact with it — issuing a `curl` against a server started two blocks earlier, for instance — because it continues running and listening throughout the rest of the document's execution. When the document finishes, whether by reaching its final block or by terminating early on an unrecoverable failure, MakeDo walks the registry and signals every process still tracked in it, guaranteeing the cleanup promised by G6 rather than leaving orphaned servers or daemons behind once the tool exits.

Detecting a background block by the presence of a trailing `&` is a lexical heuristic rather than a full parse of shell syntax, and it is not hard to construct pathological block content

— an `&` inside a quoted string, or a chain such as `a && b` — on which it would misclassify a block. The implementation does not currently guard against this; it is noted here as a known limitation rather than resolved.

3.10.5 Divergence from the formal model

The implementation realises every directive defined in Section 3.7, including `out`, `outr`, `cmd`, `pwd`, `checkpath`, and `exit`. The current directive set also includes `skip`, present in the implementation as a hard opt-out that excludes a block from execution, testing, and embedding entirely, but absent from Section 3.7 — it has no corresponding evaluation function $[\![\texttt{skip}]\!]$ and is not a member of Key as defined.

4 Evaluation

4.1 Overview

Chapter 3 defined MakeDo's model and described its implementation. This chapter evaluates whether that model delivers on the claims made for it in Chapter 1. We organise the evaluation around the three research questions introduced there.

RQ1 (Prevalence) asks how often, and for how long, executable documentation diverges from the behaviour of the software it describes. §4.2 answers this independently of MakeDo, by manually establishing ground truth for commit history since 2013 of Hugo's quick-start guide: did the documented commands actually work against the software at that point in time? This gives a prevalence measurement that does not depend on any particular tool's detection capability.

RQ2 (Prevention and Impact) asks how treating documentation as an executable test suite can prevent drift, and how intrusive its integration is into existing documentation. §4.3 answers this by evaluating MakeDo on an actively maintained project in its current state, where MakeDo located an existing, previously unreported failure in ripgrep's `GUIDE.md`, identified the exact commit that introduced it, and led to an open issue. upstream.

RQ3 (Generality) asks about the capabilities and structural limitations of applying MakeDo's documentation-centric execution model to general command-line integration testing. §4.4 answers this by porting Hugo's own integration test suite to Markdown and comparing coverage and expressiveness against the original.

4.2 Case Study 1: Prevalence of Drift in Hugo's Quick-Start Guide

4.2.1 Motivation and Methodology

To evaluate RQ1 we needed a documentation history long enough, and public enough, to measure drift over years rather than months. Hugo is a static site generator that converts Markdown into a website, and is itself widely used to build documentation sites — among its users are Kubernetes, NVIDIA, Cloudflare, and Hugo's own documentation. Hugo's quick-start guide[1] has existed continuously since Hugo's 0.9 release in 2013; at the time of writing Hugo is at release 0.165. This perfect time span, combined with a complete public commit history for both the guide and the software it documents, made it a suitable subject for a longitudinal prevalence study.

This case study is deliberately independent of MakeDo. Its purpose is to answer RQ1 on its own terms — how often, and for how long, did this guide actually diverge from the

[1] https://gohugo.io/getting-started/quick-start/

intended behaviour — without that answer being shaped by the detection capability of any particular tool. MakeDo's ability to catch these same breakages is evaluated separately, in §4.2.3.

Our procedure was as follows. We checked out the commit tagged `0.9`, rather than cloning Hugo's current `master` branch as the guide itself instructs, to guarantee the software under test matched the guide's original target exactly. Starting from this baseline, we manually executed the guide's commands against the corresponding state of the Hugo repository at every subsequent commit that touched `quickstart.md`, and recorded whether the guide's instructions still produced a working result. Where a commit introduced a divergence, we identified the specific command that failed and the reason, and continued forward until we located the commit that restored correct behaviour. This gives a binary ground-truth label for every point in the guide's history, indicating whether following the guide as written would have worked.

This case study focuses on commits that plausibly affected the guide's correctness, rather than every commit that touched `quickstart.md`. We identified candidate commits by searching commit messages and diffs for keywords associated with corrections — `fix`, `broken`, `error`, `typo`, and similar terms — and inspected each candidate manually to confirm whether it introduced or resolved an actual divergence between the guide and Hugo's behavior. This is a targeted search, not an exhaustive replay of every commit in the guide's history against a live Hugo build at that commit. It is consequently a conservative methodology: a commit that silently introduced a breakage without describing it as a fix in its message, or one we did not think to search for, would not appear in our results. The counts and durations reported in §4.2.2 should therefore be read as a lower bound on the true extent of drift in this guide's history, not a complete accounting of it.

4.2.2 Results: Six Breakages, 798 Days Broken

We identified six distinct periods during which the guide was broken. Table 4.1 summarises them.

Cause	Introduced	Fixed	Commits	Duration
Flag-parser syntax change	17 Oct 2013	23 Nov 2013	47	37 days
SSH clone URL	10 Mar 2016	17 May 2016	237	68 days
Missing `cd` before server	10 Mar 2016	22 Apr 2016	214	43 days
Markdown indentation error	27 May 2014	2 Oct 2014	152	128 days
Stray shell prompt	27 Nov 2015	29 Nov 2015	6	2 days
Unpinned theme dependency	15 Jun 2021	17 Nov 2022	905	520 days

Table 4.1: Six breakages found in Hugo's `quickstart.md` between 2013 and 2022, ordered by first occurrence. Duration is measured from the commit that introduced the breakage to the commit that fixed it.

1. **The flag-parser breakage.** The guide's first command, `hugo server --source ./docs`, worked at release 0.9. Commit `678ddef46` (17 Oct 2013) changed the flag-

parsing library Hugo relied on, `ogier/pflag`, which stopped accepting a value for `--source` separated by a space and required either `-source ./docs` or `--source=./docs` instead. Every reader following the guide from this commit onward hit the following error on the very first command:

```
2026/07/01 10:46:11 Fatal Error: flag needs an argument: --source
Usage of hugo:
  -b, --base-url="": hostname (and path) to the root eg. http://spf13.com/
  -D, --build-drafts=false: include content marked as draft
      --config="": config file (default is path/config.yaml|json|toml)
  -d, --destination="": filesystem path to write files to
  -s, --source="": filesystem path to read files relative from
      --stepAnalysis=false: display memory and timing of different steps
      --uglyurls=false: if true, use /filename.html instead of /filename/
  -v, --verbose=false: verbose output
  -w, --watch=false: watch filesystem for changes and recreate as needed
```

Commit `b351731f7` (23 Nov 2013) fixed this by switching the guide to the `--source=./docs` form. Broken for 47 commits, about 37 days. Hugo 0.9 itself shipped on 16 November 2013, one week before the fix, meaning every user who tried the guide in its first week could not get past the first command.

2. **The SSH clone failure.** Commit `6cb032b67` (10 Mar 2016) instructed readers to run `git clone git@github.com:dim0627/hugo_theme_robust.git`. This URL uses Git's SSH protocol, which requires the reader to already have an SSH key generated and registered with their GitHub account. A reader without one received `Permission denied (publickey)` and could go no further. Commit `9aa70af1a` (17 May 2016) replaced this with an HTTPS URL, which requires no such prior setup. Broken for 237 commits, about 68 days.

3. **The missing directory change.** The same commit, `6cb032b67`, told readers to `cd themes`, clone a theme into it, and then run `hugo server` without first leaving that directory, so the server could not locate the site's configuration. Commit `fbca53ac3` (22 Apr 2016) inserted a `cd ..` step before the server command. Broken for 214 commits, about 43 days.

4. **The Markdown indentation error.** Commit `a87f171bd` (27 May 2014) indented an explanatory sentence by four spaces, which CommonMark interprets as a code block; the sentence was absorbed into the preceding `git clone` fenced block. A reader who copied the whole block received a shell syntax error. Commit `c739ec228` (2 Oct 2014) removed the indentation. Broken for 152 commits, about 128 days.

5. **The erroneous prompt.** Commit `37676a25c` (27 Nov 2015) added a sample of Hugo's rebuild output to the guide, but left the line prefixed with a shell prompt: `$ Change detected, rebuilding site`. A reader who copied this line as a command received `command not found: $`. Commit `cf732940a` (29 Nov 2015) removed the

prompt. Broken for 6 commits, about 2 days.

6. **The unpinned theme dependency.** The guide instructed users to clone the master branch of the `gohugo-theme-ananke` theme and install Hugo's standard build. A subsequent change in the theme's own repository altered its styling pipeline in a way that required Hugo's extended build to work correctly. Because the guide never pinned the theme to a fixed commit, nor instructed readers to install the extended build, every reader who followed it from that point on hit a build failure when running `hugo server -D`. The fix eventually updated the guide to require the extended build and added a check for this requirement in `hugo version`'s output. This was the longest-lived failure of the six, broken for 905 commits (around 520 days), since the root cause originated outside the Hugo repository entirely.

Across these six breakages, the guide was in a broken state for 1,561 commits and approximately 798 days combined. Five of the six originated inside the Hugo repository itself, as small and individually avoidable errors: an unescaped flag, a missing `cd`, four stray spaces, a leftover prompt character. The sixth originated entirely outside Hugo's repository, in a themed dependency the guide neither pinned nor version-checked, and consequently went undetected for over a year.

4.2.3 Discussion

Documentation drift in `quickstart.md` was not a one-off incident: it recurred six times over nine years, for a combined 798 days, in one of the most widely used static-site generators in active maintenance, with no shortage of maintainer attention. Every one of the six causes — a flag syntax change, an unreachable clone URL, a missing directory change, an indentation error, a stray prompt character, and an unpinned external dependency — is the kind of failure Chapter 2 identified as invisible to purely textual consistency checking, since none of them involve a renamed identifier or a reference to code that no longer exists. Each was only observable by actually running the documented commands.

The sixth breakage is the only one of the six that originated outside Hugo's own repository, and it was also, by a wide margin, the longest-lived: 520 days against a next-longest of 128. This is consistent with the intuition that drift caused by an external dependency is systematically harder to catch than drift caused by a change inside the documented project's own commit history, since no amount of review of `quickstart.md`'s own diffs would have surfaced it; only re-running the guide against the theme's current state would. This suggests that CI triggered only by changes to the codebase is not sufficient to catch this class of documentation failure; a weekly scheduled check, for instance, independent of any commit activity — would be needed to detect it.

The SSH clone failure merits a different kind of discussion. Unlike the other five, it is not straightforwardly a bug: `git clone git@github.com:...` is a correct, working command for any reader who already has an SSH key configured with GitHub, and only fails for a reader who does not. Whether this line is ”broken” therefore depends on an assumption

about the reader's machine that the guide itself never states. This is a genuine limit on what any tool operating purely on the documentation and the software it describes can determine: MakeDo, like any executable-documentation tool, can only tell you whether a command succeeds in its own environment — not whether it will succeed in the reader's. Correctness here depends on both the command and the reader's setup, not the command alone. A project might use MakeDo to surface exactly this class of failure by running documentation checks in an environment deliberately modelling a new reader's machine — for instance, a minimal container image with no SSH agent configured — rather than the maintainer's own, already-configured development environment. This is a decision for whoever integrates MakeDo into a project's workflow to make — it is not something the tool can resolve on its own.

4.3 Case Study 2: Diagnosing Live Drift in ripgrep's Guide

4.3.1 Motivation and Methodology

The Hugo case study (§4.2) established that documentation drift is prevalent by manually reconstructing ground truth after the fact, deliberately without MakeDo's involvement. This case study addresses the complementary half of RQ2: how MakeDo, run as a user would actually run it, detects and localises drift in a real, currently maintained project, without the benefit of knowing beforehand what was broken or when it broke.

We selected ripgrep[2], a widely used command-line search tool, and its standalone user guide, `GUIDE.md`. ripgrep is a mature, actively maintained project with an established user base, and its guide is written in the same prose-plus-shell-transcript style common to the READMEs surveyed in Chapter 1. This made it a realistic subject: unlike the Hugo case study, we approached `GUIDE.md` with no prior knowledge of which examples, if any, still worked.

4.3.2 Converting the Guide with Minimal Intrusion

A design goal of MakeDo is that adopting it should not require rewriting documentation into a new format (G1, §3.2). We used this conversion as a direct test of that goal: how small a diff is required to turn an existing, unmodified guide into a working MakeDo test suite.

`makedo test` only executes fenced code blocks tagged with a supported shell language identifier (§3.4); plain, untagged fenced blocks are parsed as ordinary Markdown and never run. We leveraged this behavior as a passive filtering mechanism. Since GUIDE.md includes hypothetical examples and ones needing external tools with no setup steps, simply leaving their fences untagged was enough to exclude them from execution. This approach allowed us to leave the guide's text completely untouched.

Of the guide's code blocks, we upgraded exactly 19 from a plain fence to a `bash` fence, marking them for execution.

[2] https://github.com/BurntSushi/ripgrep

```
-```
+```bash
```

These 19 included one environment-setup block (downloading and unpacking a pinned ripgrep source release with `curl` and `unzip`), twelve ordinary worked examples already shown to produce specific output in the guide's prose (for instance, `rg fast README.md`), and six examples built around a specific dependency, `lexopt`, used throughout the guide's section on file-type filtering.

4.3.3 A Negative Assertion: Proving an Empty Result

While converting the guide, we also introduced a custom example to test MakeDo's directive model in its own right. In the guide's section on glob filtering, a working example is presented alongside prose explaining that later globs override earlier ones. To test MakeDo's ability to verify this textual claim, we created a deliberate variation of the guide's command by swapping the order of its `-g` flags:

```
$ rg lexopt -g '!*.toml' -g '*.toml'
```

Because the later glob now overrides the earlier one, the two globs cancel out and the command should match nothing at all — an explicit claim about the *absence* of output, which is harder to state as a positive assertion than as a negative one. We expressed this expected failure directly with an `exit` directive:

```
```bash
$ rg lexopt -g '!*.toml' -g '*.toml'
```
<!-- exit 2 -->
```

ripgrep returns exit status 2 when a search produces no matches, so this directive is a direct, minimal encoding of the behaviour described in the prose. With no `out` directive present, MakeDo makes no assertion about output content at all, and the block passes purely on the strength of the exit code matching what the guide already promises. This is a small example, but it demonstrates something the `out`/`outr` directives alone cannot express cleanly: a claim about a command's absence of behaviour, rather than the presence of specific text (G3, §3.2).

4.3.4 Finding and Explaining the Failure

Running `makedo test` against the converted guide produced the output shown in Listing 4.1:

```
FAIL GUIDE.md:277
| rg lexopt
| exit 1

[...]

X 14/20 tests passed - 6 block(s) failed
Error: 6 test(s) failed
```

Listing 4.1: The output of running `makedo test` against the converted guide.

The 14 passing blocks include the twelve ordinary worked examples, the environment-setup block, and the negative-assertion block described above. The six failures are the remaining six `lexopt` examples. This result is not merely a MakeDo artefact: following the guide by hand, exactly as written, produces the same errors. A reader who downloads release 0.7.1 as the guide instructs and then runs any of the six `lexopt` examples finds nothing, because `lexopt` is not a dependency of that release at all.

The reason was straightforward to identify once we looked at `GUIDE.md`'s own history. Ripgrep 0.7.1 used `clap`[3] for its command-line argument parsing at the time the guide's setup instructions were written, and the guide's filtering examples originally searched for `clap` accordingly. The guide's filtering section was later rewritten to search for `lexopt`[4] instead, in commit `c28ce76`, reflecting a dependency ripgrep adopted in later releases.

```
$ curl -LO https://github.com/BurntSushi/ripgrep/releases/.../ripgrep-0.7.1-....
    tar.gz

...

$ rg lexopt -g '*.rs'
```

Listing 4.2: The incompatible setup instructions and filtering example from ripgrep's `GUIDE.md`.

As shown in Listing 4.2, this change updated the guide's examples but not its setup instructions, which still direct readers to download release 0.7.1 — a release that predates `lexopt` entirely. From `c28ce76` onward, the guide's setup step and its filtering examples describe two different, mutually incompatible states of the ripgrep source tree.

This is a different shape of drift to any of the six found in the Hugo study (§4.2.2). `lexopt` is not a typo, a removed feature, or a broken link: it is a real string, present in ripgrep's current source code, and the guide's prose describing it is accurate for a recent checkout of ripgrep. The mismatch is that the guide's own setup instructs readers to download an old version of the codebase. A tool comparing the guide's text against ripgrep's current codebase would find `lexopt` exactly where the guide says it should be, and report nothing

[3] https://crates.io/crates/clap
[4] https://crates.io/crates/lexopt

wrong; only running the guide's own instructions, in order, against the exact version they specify, surfaces the failure.

4.3.5 Reproducing Detection in CI

§4.3.4 establishes that MakeDo detects this drift as it exists today, and identifies the commit that introduced it. To test whether MakeDo could have caught the drift *before* it reached readers, rather than only after the fact, we forked ripgrep and reconstructed the moment `c28ce76` was introduced as a CI event.

We first created a commit[5] on the fork restoring the guide's original, pre-`c28ce76` state, in which the filtering examples search for `clap` rather than `lexopt`, consistent with the release 0.7.1 the setup step downloads. This commit also applied the MakeDo conversion described in §4.3.2, including the `exit 2` negative-assertion block from §4.3.3, and added a CI workflow running `makedo test` against `GUIDE.md` on every push. This workflow ran[6] and passed: with the guide in its pre-`c28ce76` state, all executable blocks, including the negative assertion, succeeded against the release 0.7.1 source tree.

We then applied the same change `c28ce76` made upstream — rewriting the guide's filtering examples to search for `lexopt` in place of `clap`, while leaving the setup step, the rest of the guide, and the negative-assertion block untouched — and pushed this as a second commit to the same fork. The CI workflow ran again[7] and failed, reporting the same six block failures identified in §4.3.4.

This reconstructs, commit for commit, the exact change that introduced the drift in the real ripgrep repository, and shows that a CI check using MakeDo passes immediately before that change and fails immediately after it. This demonstrates the mechanism MakeDo is intended to support in practice (G1, §3.2): rather than relying on periodic audits or manual replay, a project can run `makedo test` as an ordinary CI check on documentation changes. Had ripgrep's own repository run such a check at the time, `c28ce76` would have failed CI in its original form, and the six-block failure identified in §4.3.4 would never have reached a reader.

4.3.6 Results Summary

Table 4.2 summarises the outcome of this case study.

This failure had not been previously reported. We opened an issue on ripgrep's GitHub repository[8], explaining that the guide's setup instructions and its filtering examples now target different, mutually incompatible states of the ripgrep source tree, and proposing two possible fixes: updating the guide's examples back to `clap` to match release 0.7.1, or updating the guide's setup instructions to point at a release recent enough to include `lexopt`. At the time of writing the issue remains open.

[5] https://github.com/MarlonBando/ripgrep-makedo-studycase/commit/3f63c5da75541275356607735cefa51f7f102d9

[6] https://github.com/MarlonBando/ripgrep-makedo-studycase/actions/runs/29747033677

[7] https://github.com/MarlonBando/ripgrep-makedo-studycase/actions/runs/29748029732/job/88370917188

[8] https://github.com/BurntSushi/ripgrep/issues/3448

Step	Outcome
Convert `GUIDE.md`	20 of the guide's code blocks marked executable; all other blocks left untouched
Run `makedo test`	14/20 blocks pass, 6 fail
Manual verification	Same 6 examples fail when followed by hand
Root cause	Guide's setup step targets release 0.7.1 (uses `clap`); guide's examples target `lexopt`, introduced in a later release
Commit identified	`c28ce76`
CI reconstruction	Pre-`c28ce76` commit: MakeDo CI passes (run 29747033677). `c28ce76` replayed: MakeDo CI fails (run 29748029732)
Upstream report	Issue #3448 opened on ripgrep's repository, open at time of writing

Table 4.2: Summary of the ripgrep `GUIDE.md` case study.

4.3.7 Discussion

This case study addresses RQ2 under conditions the Hugo case study deliberately did not: no prior knowledge of what was broken, in a project we had no involvement in maintaining, using MakeDo exactly as a maintainer or contributor would. The result is a genuine, previously unreported instance of documentation drift in an actively maintained, widely used tool, found by converting the guide with a minimal diff and running `makedo test` once.

The nature of this drift is also instructive. Every one of the Hugo breakages (§4.2.2) originated from a single change — a flag, a URL, an indentation, a missing dependency pin — at a single point in time. The ripgrep case is a compound failure: two separate parts of the same guide, the setup instructions and the filtering examples, drifted out of agreement with each other rather than either one becoming individually wrong. Neither half of the guide is false in isolation — release 0.7.1 is a real, installable release of ripgrep, and `lexopt` is a real dependency of current ripgrep — the failure exists only in their combination. It is also a second, independent instance of exactly the phenomenon RQ1 measured in the Hugo study (§4.2): documentation drift, present and unreported, in a mature, actively maintained open-source project, which strengthens the prevalence claim beyond a single case study.

The CI reconstruction in §4.3.5 addresses a limitation left open by the Hugo case study: retrospective detection alone does not establish that MakeDo would function usefully as part of a project's actual workflow. Reproducing the exact moment `c28ce76` was introduced, commit for commit, and showing that a MakeDo CI check transitions from passing to failing at precisely that point, is stronger evidence for RQ2 than detection after the fact alone, since it demonstrates prevention rather than only diagnosis.

We note two limitations. First, the negative-assertion example in §4.3.3, while a useful

demonstration of (G3, §3.2), is a single instance and not itself evidence of prevalence; it shows directive expressiveness but in this case did not catch any drift. Second, the CI workflow used in §4.3.5 was configured by us, on a fork, rather than integrated into ripgrep's actual continuous integration; the reconstruction demonstrates that the mechanism functions as intended, but the guide has not, at the time of writing, actually been protected by it in ripgrep's own repository.

4.4 Case Study 3: General Integration Testing in Hugo

4.4.1 Motivation and Methodology

The first two case studies evaluated MakeDo as a tool for executable documentation. RQ3 asks a broader question: what capabilities and structural limitations emerge when applying this documentation-centric execution model to general command-line integration testing.

Rather than constructing examples specifically for this evaluation, we applied MakeDo to Hugo's existing integration test suite. Hugo tests itself using `testscript`[9], a mature testing framework extracted from the Go toolchain. The suite contains command-line tests, configuration tests, and long-running development-server tests, making it representative of the kinds of workflows MakeDo was designed to support.

Our evaluation consisted of two experiments:

First, we attempted to port Hugo's complete `testscript` integration suite to MakeDo without modifying Hugo itself. This measures the expressiveness of MakeDo's execution model: how much of an existing integration suite can be represented using only Markdown documents, shell commands, and MakeDo directives.

Second, we evaluated whether the resulting tests detect behavioural regressions comparably to the original suite. We introduced ten small, independent mutations into Hugo's source code, compiled a separate binary for each mutation, and executed both the original `testscript` suite and the ported MakeDo suite against every mutated binary.

To make this experiment reproducible, every injected mutation was committed individually in a public fork of Hugo.[10] Each commit introduces exactly one behavioural change, allowing the evaluation to be reproduced by checking out the corresponding commit and running either test suite.

4.4.2 Porting the Test Suite

The original Hugo integration suite contains 75 test cases distributed across five directories. We successfully ported 70 of them, a coverage of 93%.

The five omitted tests all rely on Windows-specific conditional execution. MakeDo currently has no mechanism for expressing platform-dependent execution within a Markdown

[9] https://pkg.go.dev/github.com/rogpeppe/go-internal/testscript
[10] https://github.com/MarlonBando/hugo/commits/master

document, making these tests impossible to represent without extending the directive language. Apart from this limitation, every remaining test ported directly.

The resulting MakeDo suite exercises the same behaviours as the original suite, including command-line argument handling, configuration loading, file generation, and Hugo's long-running development server. Background server tests require no additional framework support: the server is started as a background process, MakeDo waits until its readiness directive succeeds, subsequent blocks interact with the running server, and the process is terminated automatically once the document finishes. This behaviour required several custom helper functions in Hugo's original `testscript` implementation but follows directly from MakeDo's block lifecycle model.

Porting the suite also provides a useful comparison of testing complexity. Table 4.3 compares both suites, including their supporting framework code.

	testscript	**MakeDo**
Test definitions	2,330 lines	5,252 lines
Framework helpers	352 lines (Go)	12 lines (Bash)
Total	2,682 lines	5,264 lines

Table 4.3: Approximate size of the Hugo integration suites. Framework helpers include custom commands required by each framework but not written directly inside the tests themselves.

The MakeDo suite is therefore substantially more verbose than the original. This increase comes from two deliberate design choices. Rather than embedding filesystem state inside a custom archive format such as `txtar`, MakeDo creates files explicitly using ordinary shell commands, and expected behaviour is written directly alongside the commands using Markdown directives such as `<!-- out -->`.

This verbosity also has an advantage. Nearly all of the testing logic is visible inside the Markdown documents themselves. In contrast, `testscript` relies on over 350 lines of Go helper functions that implement custom commands such as `httpget`, `waitServer`, `stopServer`, and `replace`. These commands simplify the scripts but require readers to understand both the test language and its supporting implementation. MakeDo instead keeps its tests almost entirely self-contained, relying only on standard shell syntax and a small shell wrapper used for polling server responses. An example comparing a ported MakeDo test to its original `testscript` equivalent can be found in Appendix E.

4.4.3 Mutation Testing

Coverage alone does not establish whether the ported tests are capable of detecting real regressions. To evaluate this directly, we introduced ten independent mutations into Hugo's source code and compared the behaviour of the original `testscript` suite with the ported MakeDo suite.

The mutations represent small but realistic implementation mistakes, including reversed

conditionals, altered default values, incorrect output strings, and changes affecting Hugo's development server. Each mutation was introduced independently and evaluated against a freshly compiled binary.

Table 4.4 summarises the results.

Mutation	Behaviour Changed	testscript	MakeDo
`env_goos`	Incorrect operating system label	Not caught	Not caught
`list_csv_header`	Incorrect CSV column header	Not caught	Not caught
`list_drafts`	Draft filtering logic	Caught	Caught
`list_future`	Future content filtering	Caught	Caught
`list_expired`	Expired content filtering	Caught	Caught
`server_watch`	File watching disabled	Caught	Caught
`server_live_reload`	Live reload disabled	Caught	Caught
`server_fast_render`	Fast render mode disabled	Caught	Caught
`env_version`	Version string replaced	Caught	Caught
`server_append_port`	Incorrect base URL generation	Caught	Caught
Total		**8/10**	**8/10**

Table 4.4: Mutation testing results comparing Hugo's original `testscript` integration suite with the ported MakeDo suite.

Both frameworks detected the same eight mutations and missed the same two. The two undetected mutations modified output that neither suite asserted upon, indicating gaps in the original test coverage rather than limitations specific to MakeDo.

The remaining eight mutations include every mutation affecting Hugo's development server. These require observing the behaviour of a long-running background process rather than the output of a single command, providing a direct evaluation of MakeDo's background-process execution model. On these mutations, MakeDo matched the behaviour of the original `testscript` suite exactly.

4.4.4 A Structural Limitation: Output Streams

Although mutation testing showed identical detection rates for the ten mutations evaluated, it does not establish that both frameworks are equivalent. We therefore examined a known behavioural difference between their execution models by replaying two regressions from Hugo's own integration test history.

The first concerns Hugo's logging behaviour. Commit `9dfa11261`[11] changed Hugo so that all log output (`INFO`, `WARN`, and `ERROR`) was written to `stderr` rather than `stdout`. Hugo's corresponding `testscript` assertion had to be updated from `stdout` to `stderr` to keep the test passing.

The second concerns the exact text of diagnostic output. Commit `9197debbf`[12] relaxed a

[11] https://github.com/gohugoio/hugo/commit/9dfa1126177952433f8339a87fb566c2959fc3d6
[12] https://github.com/gohugoio/hugo/commit/9197debbfef3f321e6b5f74f0d783377acbfcca0

`testscript` assertion after Hugo changed the prefix of a deprecation message from `INFO` to `ERROR`. The original assertion matched the complete message and therefore failed after the change; the revised test instead matches only the stable portion of the warning.

We recreated both situations by restoring the original assertions and executing them against the current Hugo binary using both `testscript` and MakeDo.

The original `testscript` suite correctly detects both regressions because it treats `stdout` and `stderr` as independent streams and allows assertions over the exact output. MakeDo does not. Commands execute inside a pseudo-terminal so that they behave exactly as they would for an interactive user, and the terminal presents a single multiplexed output stream. Consequently, `out` and `outr` directives cannot distinguish whether a message originated from `stdout` or `stderr`. Likewise, the `outr` directive succeeds as long as the specified substring appears somewhere in the combined output, making it less sensitive to changes such as modified log prefixes.

These behaviours follow directly from MakeDo's design. For executable documentation, readers observe terminal output rather than separate file descriptors, making a combined output stream the appropriate abstraction. However, when MakeDo is used as a general integration testing framework, this abstraction prevents it from detecting certain regressions that depend either on the output stream carrying a message or on precise changes to diagnostic output.

4.4.5 Results Summary

Table 4.5 summarises the principal findings.

Measure	Result
Test cases attempted	75
Test cases ported	70 (93%)
Testsuite size	2,682 lines (`testscript`) vs 5,264 lines (MakeDo)
Framework helpers	352 lines (Go) vs 12 lines (Bash)
Mutations detected	8/10 by both frameworks
Historical stream regressions	`testscript`: 2/2; MakeDo: 0/2

Table 4.5: Summary of the Hugo integration-testing case study.

4.4.6 Discussion

This case study provides evidence for RQ3 from three complementary perspectives.

First, MakeDo proved expressive enough to represent the large majority of a mature integration suite that it was not designed for. Of Hugo's 75 integration tests, 70 ported directly. The five omitted tests all depend on platform-conditional execution, a capability outside MakeDo's current execution model rather than a limitation of Markdown itself.

Second, the mutation study shows that this expressiveness translates into comparable fault detection. Across ten independently designed mutations, MakeDo detected exactly the same eight behavioural regressions as Hugo's original `testscript` suite, including

every mutation affecting the long-running development server. Although the sample size is necessarily limited, these results suggest that MakeDo's directive model is capable of expressing realistic integration tests rather than merely replaying command output.

Third, the comparison highlights an important trade-off in MakeDo's design. The ported suite is substantially more verbose than the original because MakeDo expresses filesystem setup, expected output, and command execution explicitly within Markdown documents. This increases the size of the test definitions but also makes the testing logic almost entirely self-contained. Unlike `testscript`, which relies on several hundred lines of hidden framework code implementing custom commands, MakeDo requires only a small shell wrapper and otherwise relies on ordinary shell syntax already familiar to its users.

The evaluation also identifies two important limitations. The first is the lack of platform-conditional execution, which prevented five Windows-specific tests from being ported. The second is MakeDo's merged output model. Because commands execute inside a pseudo-terminal, `stdout` and `stderr` are intentionally multiplexed before directive evaluation. This accurately models what an interactive user observes but prevents MakeDo from detecting regressions that depend on which stream a message was written to.

Finally, the mutation study itself has a limitation. The evaluation uses ten manually designed mutations rather than a large automatically generated mutation corpus. The reported parity therefore demonstrates comparable behaviour on this representative sample rather than proving equivalence between the two frameworks.

Overall, this case study comprehensively answers RQ3. MakeDo successfully ports the large majority of a mature integration suite, detects behavioural regressions at the same rate as the original framework on the evaluated mutations, and does so using ordinary Markdown and shell syntax. The identified limitations are specific and well understood, rather than broad shortcomings of the execution model, and arise directly from design decisions made to support executable documentation.

4.5 Summary of Findings

The three case studies presented in this chapter address the research questions introduced in Chapter 1. This section draws together their results to assess MakeDo's execution model as a whole.

4.5.1 RQ1 (Prevalence)

The investigation into Hugo's quick-start guide (§4.2) showed that documentation drift is a persistent phenomenon rather than a rare anomaly. Over a nine-year period, the guide fell out of sync with Hugo's actual behaviour six separate times, accumulating 798 days during which readers following the guide encountered broken instructions. This occurred despite Hugo being an actively maintained, highly visible project. Crucially, these breakages arose while the documentation itself remained textually plausible, illustrating forms of semantic drift that require execution-based validation rather than purely textual consistency checking to detect reliably.

4.5.2 RQ2 (Prevention and Impact)

Together, the retrospective Hugo evaluation and the live ripgrep case study demonstrated that MakeDo's execution model can both diagnose historical documentation drift and detect previously unreported failures in actively maintained projects. Running MakeDo against ripgrep's guide, without prior knowledge of any breakages, immediately surfaced a compound failure where the setup instructions and worked examples had drifted out of agreement. Reconstructing the historical commit that introduced this drift further showed that integrating MakeDo into a continuous integration workflow would have caused the change to fail before reaching users. Furthermore, applying this model proved highly non-intrusive, requiring only the upgrade of existing Markdown code fences without altering the guide's prose. These results provide evidence that MakeDo's execution model is capable of supporting documentation validation as part of an ordinary development workflow.

4.5.3 RQ3 (Generality)

The conversion of Hugo's integration tests (§4.4) showed that MakeDo's execution model is applicable beyond documentation validation as a framework for command-line integration testing. MakeDo successfully ported 70 of Hugo's 75 integration tests, demonstrating that its directive model is expressive enough to represent the large majority of an existing test suite. On the evaluated mutation sample, the ported MakeDo suite detected exactly the same behavioural regressions as Hugo's original `testscript` suite. These results suggest that ordinary Markdown, combined with MakeDo's directives, is capable of expressing realistic command-line integration tests while exposing well-defined limitations in platform-conditional execution and stream-sensitive assertions.

4.5.4 Overall Implications

Taken together, the three case studies provide evidence that treating Markdown as an executable artefact is both practical and useful. Across two independent documentation studies, executable Markdown exposed forms of documentation drift that would not have been detected through textual analysis alone. A third case study further demonstrated that the same execution model can express the large majority of an existing command-line integration suite without requiring a dedicated testing language.

The evaluation also identifies clear boundaries to the current design. Platform-conditional execution remains outside MakeDo's execution model, merged terminal output prevents assertions over individual output streams, and the mutation study is necessarily limited to a relatively small manually designed sample. These limitations are specific and well-understood, providing concrete directions for future work rather than broad shortcomings of the overall approach.

Overall, the evaluation supports the central claim of this thesis: embedding executable assertions directly within Markdown provides a practical mechanism for reducing documentation drift while also enabling the same documents and execution model to be applied to command-line integration testing.

5 Conclusion

5.1 Summary

Software documentation serves as the primary bridge between a system and its users. When that documentation takes the form of command-line tutorials, installation guides, or READMEs, readers expect the instructions to execute exactly as described. Yet, as software evolves, its documentation inevitably drifts from reality. Traditional consistency-checking approaches rely primarily on static textual analysis, catching broken links or renamed identifiers while missing the semantic drift that occurs when command-line behaviour changes, output formats evolve, or external dependencies introduce incompatibilities. The result is documentation that remains syntactically plausible while failing in practice.

This thesis presented MakeDo, an execution model that treats ordinary Markdown as an executable artefact. Rather than requiring authors to rewrite documentation in a dedicated testing language, MakeDo embeds assertions directly within Markdown using lightweight directives. By maintaining shell state across code blocks, supporting long-running background processes, and providing typed placeholders for non-deterministic values, the model enables realistic command-line workflows to be validated while preserving the readability of the documentation itself.

The evaluation presented in Chapter 4 addressed the three research questions introduced in Chapter 1. The longitudinal study of Hugo's Quick-Start guide showed that documentation drift is a recurring phenomenon, demonstrating that the instructions were broken and unusable for a total of 798 days over the nine years analysed. Applying MakeDo to ripgrep's user guide surfaced a previously unreported documentation failure, and reconstructing the commit that introduced the regression demonstrated that a CI workflow using MakeDo would have rejected the change before it reached users. Finally, porting Hugo's integration test suite showed that the same execution model can be applied beyond documentation validation: MakeDo successfully expressed 70 of Hugo's 75 integration tests and matched the original `testscript` suite's mutation-detection rate on the evaluated mutation sample.

Taken together, these results provide evidence that the three research questions can be answered positively. Documentation drift is both prevalent and long-lived in real software projects; executable Markdown can detect and localise such drift in practice with minimal intrusion; and the same execution model is expressive enough to support a substantial class of command-line integration tests in addition to documentation validation.

5.2 Limitations

The evaluation also identified several limitations of the current design.

First, because MakeDo executes commands within a pseudo-terminal to model an interactive terminal session, standard output (`stdout`) and standard error (`stderr`) are merged before they are observed. This accurately reflects what a documentation reader sees but prevents assertions about which output stream carried a particular message. As demonstrated in the Hugo case study, this means that regressions involving stream redirection cannot currently be detected.

Second, MakeDo does not currently support platform-conditional execution. Tests that depend on operating-system-specific behaviour, such as Windows-only commands or filesystem semantics, cannot be expressed directly. This limitation prevented the porting of five tests from Hugo's integration suite.

Third, executable documentation can only validate behaviour within the environment in which it executes. Whether a documented command succeeds may depend on assumptions about the reader's machine that are external to the documentation itself, such as the presence of SSH credentials or other locally configured resources. These assumptions must therefore be modelled explicitly by whoever integrates MakeDo into their testing workflow.

Finally, the mutation study was intentionally limited to a relatively small set of manually designed mutations within a single project. While this was sufficient to compare MakeDo with Hugo's existing integration tests, a larger empirical study across multiple projects would provide stronger evidence for the generality of the approach.

5.3 Future Work

Several avenues remain for extending MakeDo.

One priority is extending the execution model to support simulated user input and retroactive assertions over long-running background processes. Many real-world command-line workflows are not simple request/response interactions: a background server's behaviour may change as a result of a separate, later action (a file being modified, a second process connecting, a configuration being reloaded), and documentation often assumes a reader is able to type responses into an interactive prompt. Supporting scripted input alongside the ability to assert on the state or output of a background process at a later point in the execution timeline would allow MakeDo to express this class of workflow, bringing the model considerably closer to the way real documentation and real production systems actually behave.

A second direction concerns output handling. Supporting assertions over individual output streams without sacrificing the pseudo-terminal model would broaden MakeDo's applicability to integration testing while preserving its suitability for executable documentation. Exploring this trade-off remains an open design question.

Beyond the execution model itself, a larger empirical evaluation would strengthen the evidence presented in this thesis. The case studies in this work were sufficient to vali-

date the MakeDo model and to demonstrate its practical value, but they remain limited in number and scope, and may not be representative of the full range of behaviour found across real-world projects. Applying MakeDo more broadly – analysing the documentation history of additional projects in the same longitudinal manner as the Hugo case study, and evaluating a wider range of documentation styles and existing integration test suites – would provide a more comprehensive assessment of its expressiveness and practical limitations.

5.4 Concluding Remarks

This thesis has shown that ordinary Markdown can serve not only as documentation but also as an executable test suite. Embedding assertions directly within documents enables documentation to be validated, reducing the opportunity for documentation drift to accumulate unnoticed as software evolves.

The evaluation demonstrates that this approach is practical for real documentation and expressive enough to be used for integration testing, while also identifying clear boundaries to the current design. Rather than replacing specialised testing frameworks, MakeDo provides a complementary mechanism for ensuring that the instructions presented to users remain consistent with the behaviour of the software they describe.

As software projects continue to rely on documentation as their primary interface with users, treating that documentation as executable rather than purely descriptive offers a practical step towards making it more reliable, easier to maintain, and better integrated into modern software development workflows.

6 Declaration of AI Usage

6.1 Development

During the development of *MakeDo*, generative AI was used extensively as a development aid. The primary tools were GitHub Copilot, available through the GitHub Student Developer Pack, and Gemini AI Pro, which was also provided through the student program. The models used throughout the project were Gemini 3.1 and Claude Sonnet.

Interaction with these models was performed through several AI coding harnesses, namely *OpenCode*, *GitHub Copilot CLI*, and *Antigravity CLI*. These harnesses provided an interactive environment for planning, implementing, and refining features throughout the development process. In addition, a custom `/grill-me` skill (Listing C.2) was used to guide the AI towards analysing implementation strategies before generating code. The complete AI agent instructions are included in Appendix C (Listing C.1), while the project-specific *MakeDo* skill is provided in Listing C.3.

For each feature, the AI was first asked to analyse the problem and discuss alternative implementation strategies using the `/grill-me` workflow. This workflow was primarily used to explore the design space, identify potential advantages and disadvantages of different approaches, and select the most appropriate implementation. Once a suitable design had been identified, the AI was instructed to generate an initial implementation. This prototype was then evaluated in practice to determine whether it fulfilled the intended requirements.

All generated code was manually reviewed before being accepted. If the generated implementation closely matched the intended design, only minor modifications were required, such as simplifying the implementation, removing unnecessary code, or improving readability and maintainability. In cases where the generated solution did not align with the desired architecture or coding style, parts of the implementation, or occasionally the entire feature, were rewritten manually.

Generative AI was also used extensively for test generation. Most Go unit tests were generated by AI and were primarily validated by executing the test suite rather than through detailed manual code review. In contrast, the Markdown-based integration and behavioural tests were always manually reviewed and were frequently rewritten or extended to better capture the intended behaviour and improve their clarity.

6.2 Writing

Furthermore, generative AI assisted during the writing of this thesis, primarily as a language assistance tool. Since English is not my first language, AI was mainly used to correct grammar, improve fluency, and enhance the overall readability of the text without altering its intended meaning.

The typical workflow consisted of writing each section manually, occasionally leaving `TODO` markers in places where more advanced LaTeX knowledge was required. Once the technical content was considered complete, AI was used to polish the writing by correcting grammatical errors, improving sentence flow, and removing awkward phrasing. Prompts explicitly instructed the AI to preserve both the meaning and the overall structure of the original text, ensuring that the technical content and arguments remained my own. Every suggested revision was reviewed against the original passage: where the AI's proposed wording was considered a clear improvement, it was retained as suggested; where it altered the intended meaning, removed content, or was not considered an improvement, the original text was kept or adjusted manually instead. As a result, some passages in this thesis reflect AI-suggested phrasing that was judged to improve clarity, while others remain unchanged from the original manual draft.

In addition to language assistance, AI was used to provide feedback on the written content. After completing a section, the AI was asked to critique the clarity of the explanations, identify passages that could be misunderstood, and suggest improvements to the overall flow and organisation. These suggestions were treated as feedback only; any revisions were evaluated and incorporated manually where appropriate.

Finally, generative AI assisted with the creation of selected LaTeX figures. For example, the LaTeX source code for Figure 3.2 was generated with AI assistance. The desired figure was first described conceptually, accompanied by a hand-drawn sketch of the intended layout. Through several iterations, the AI produced the final LaTeX code used to generate the figure.

6.3 Research

Generative AI was used as an aid during the research phase of this thesis in four main ways.

First, AI was used to support the literature review process by identifying tools and academic work related to the problem being tackled. Given a description of the specific gap or mechanism under investigation, the AI was asked to suggest existing tools, frameworks, and papers that addressed similar or related problems; these sources were then independently retrieved and evaluated to determine their relevance and suitability for citation.

Second, AI was used to verify the accuracy of citations within the text. For each citation, both the written passage and the source paper were provided to the AI, which was asked to identify any imprecisions, misrepresentations, or unsupported claims in how the source was being described. Flagged passages were then manually reviewed against the original source and corrected where necessary, ensuring that claims attributed to a paper accurately reflected its content.

Third, an agentic AI workflow was used to assist with the tool survey presented in Appendix D. The AI was provided with the full list of repositories to be surveyed, together

with precise instructions specifying what to look for within each repository (such as the presence of a particular feature or mechanism). The AI then searched each repository accordingly and reported its findings. All claims produced through this process were subsequently double-checked manually against the corresponding repository before being included in the survey to ensure their correctness.

Fourth, agentic AI was used to automate repetitive manual operations once a clear methodology was established. For instance, during the data collection for RQ1, the process of replaying historical commits was fully automated by deploying an AI agent in a loop. Once the agent was instructed on the exact steps to perform for a single commit, it independently executed those steps across the dataset, significantly accelerating the process.

Bibliography

[1] John MacFarlane. *CommonMark Specification*. https://spec.commonmark.org/. CommonMark Specification, version 0.31.2. 2024.

[2] Felipe Zlotnick. *GitHub Open Source Survey 2017*. Tech. rep. GitHub, 2017. DOI: 10.5281/zenodo.806811. URL: https://doi.org/10.5281/zenodo.806811.

[3] Wen Siang Tan, Markus Wagner, and Christoph Treude. "Detecting Outdated Code Element References in Software Repository Documentation". In: *Empirical Software Engineering* (2022). arXiv: 2212.01479 [cs.SE]. URL: https://arxiv.org/abs/2212.01479.

[4] zimbatm. *mdsh: A Markdown Shell Pre-processor*. https://github.com/zimbatm/mdsh. 2024.

[5] Stateful, Inc. *Runme: Discover and Run Code Snippets Directly from Markdown Files*. https://github.com/runmedev/runme. 2024.

[6] Python Software Foundation. *doctest — Test Interactive Python Examples*. https://docs.python.org/3/library/doctest.html.

[7] Gede Artha Azriadi Prana et al. "Categorizing the Content of GitHub README Files". In: *Empirical Software Engineering* 24 (2019), pp. 1296–1327.

[8] Yubin Liu, Ehsan Noei, and Kelly Lyons. "How ReadMe Files Are Structured in Open Source Java Projects". In: *Information and Software Technology* 148 (2022), p. 106924.

[9] Lin Tan et al. "/*iComment: Bugs or Bad Comments?*/". In: *Proceedings of the 21st ACM Symposium on Operating Systems Principles (SOSP)*. 2007, pp. 145–158. DOI: 10.1145/1294261.1294276.

[10] Shin Hwei Tan et al. "@tComment: Testing Javadoc Comments to Detect Comment-Code Inconsistencies". In: *Proceedings of the 5th IEEE International Conference on Software Testing, Verification and Validation (ICST)*. 2012, pp. 260–269.

[11] Inderjot Kaur Ratol and Martin P. Robillard. "Detecting Fragile Comments". In: *Proceedings of the 32nd IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 2017, pp. 112–122.

[12] Fengcai Wen et al. "A Large-Scale Empirical Study on Code-Comment Inconsistencies". In: *Proceedings of the 27th IEEE/ACM International Conference on Program Comprehension (ICPC)*. 2019, pp. 53–64. URL: https://ieeexplore.ieee.org/document/8813274.

[13] Donald E. Knuth. "Literate Programming". In: *The Computer Journal* 27.2 (1984), pp. 97–111.

[14] Thomas Kluyver et al. "Jupyter Notebooks – A Publishing Format for Reproducible Computational Workflows". In: *Positioning and Power in Academic Publishing: Players, Agents and Agendas (ELPUB 2016)*. IOS Press, 2016, pp. 87–90.

[15] João Felipe Pimentel et al. “Assessing and Restoring Reproducibility of Jupyter Notebooks”. In: *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 2020.

[16] João Felipe Pimentel et al. “A Large-Scale Study About Quality and Reproducibility of Jupyter Notebooks”. In: *Proceedings of the 16th International Conference on Mining Software Repositories (MSR)*. 2019.

[17] aiiie. *Cram*. https://github.com/aiiie/cram. GitHub repository.

[18] Aurelio Jargas. *clitest: Portable POSIX shell script for automatic testing of command-line interfaces*. https://github.com/aureliojargas/clitest. 2024.

[19] byexamples. *byexample: Executable Documentation*. https://github.com/byexamples/byexample. 2024.

[20] Yuin. *Goldmark: A markdown parser written in Go*. https://github.com/yuin/goldmark. GitHub repository. 2019.

A Examples of Source-Code Documentation Drift

The following examples illustrate the specific types of documentation drift targeted by the static and targeted-execution tools discussed in Section 2.2. They demonstrate inconsistencies between inline comments and implementation logic, contrasting with the execution-based drift targeted by MakeDo.

```c
/*
 * Updates the global routing table.
 * Note: The caller MUST hold the routing_mutex lock before
 * calling this function to prevent race conditions.
 */
void update_routes(route_t *new_route) {
    // DRIFT: A later maintainer added an internal lock acquisition,
    // violating the comment's rule and risking a deadlock if
    // the caller actually followed the documentation.
    mutex_lock(&routing_mutex);
    apply_route_changes(new_route);
    mutex_unlock(&routing_mutex);
}
```

Figure A.1: An example of natural-language rule drift. Tools like iComment extract implicit rules from comments (e.g., lock prerequisites) and flag when the implementation contradicts them [9].

```java
/**
 * Configures the connection pool size.
 *
 * @param poolSize the maximum number of connections.
 * @throws IllegalArgumentException if poolSize is negative.
 */
public void initPool(int poolSize) {
    // DRIFT: The code was updated to silently default to a size of 1,
    // but the Javadoc was never updated to reflect the removed exception.
    if (poolSize <= 0) {
        poolSize = 1;
    }
    this.size = poolSize;
}
```

Figure A.2: An example of Javadoc contract drift. @tComment targets this by generating tests to verify if the documented `@param` and `@throws` behaviour matches the runtime execution [10].

```
// Checks if the user is an admin before granting access.
// The isAdmin flag is populated during the initial auth handshake.
public boolean checkPermissions(User user) {
    // DRIFT: The identifier 'isAdmin' was refactored to 'hasRootAccess',
    // leaving the comment text stale and referring to a non-existent field.
    return user.hasRootAccess();
}
```

Figure A.3: An example of stale identifier drift. Tools like Fraco use part-of-speech tagging to detect when a refactored variable name leaves surrounding comments out of date [11].

B Tool Syntax Examples

This appendix provides concrete syntax examples for the tools discussed in Section 2.5, illustrating their capabilities and limitations in handling executable documentation.

B.1 `mdsh`: In-Place Output Capture

As discussed in the text, `mdsh` uses a comment-based syntax to find code blocks and rewrites the file in place to capture standard output. It does not natively parse Markdown or capture exit statuses.

Before running `mdsh`:

````
Here is an example of checking the current working directory:

<!-- mdsh -->
```bash
pwd
```
````

After running `mdsh`:

````
Here is an example of checking the current working directory:

<!-- mdsh -->
```bash
pwd
```
```output
/home/user/project
```
````

B.2 Runme: Notebook Execution and Shared State

Runme enhances standard Markdown with JSON-like annotations to manage execution state, such as naming blocks or running them in the background. It lacks a built-in assertion syntax, requiring external tools (such as Bats) for verification.

Example `README.md` using Runme annotations:

````
Start the background server and export the port to the session state:

```bash {"name": "start-server", "background": true, "interactive": false}
````
````

````
export PORT=8080
./bin/server --port $PORT
```

Ping the server to ensure it is running:

```bash {"name": "ping-server"}
curl http://localhost:$PORT/health
```
````

B.3 Cram: Transcript Comparison Testing

Cram relies on its own `.t` file format rather than Markdown. It uses a strict two-space indentation rule to distinguish tests from prose, and represents expected output and non-zero exit codes directly beneath the command.

Example `cli-test.t` file:

```
This is standard prose describing the test suite. Cram ignores unindented text.

Commands must be indented by exactly two spaces and start with a dollar sign:

  $ echo "Testing the CLI"
  Testing the CLI

When a command fails, Cram can assert the specific exit status using brackets:

  $ cat non_existent_file.txt
  cat: non_existent_file.txt: No such file or directory
  [1]
```

B.4 `clitest`: Inline Markdown Assertions

`clitest` operates directly on Markdown files, utilizing inline comments to assert expected standard output and exit codes. It relies on line-matching rather than a Markdown parser.

Example Markdown block tested by `clitest`:

````
The application should gracefully fail when missing a configuration file:

```bash
./app --config missing.yml
#=> Error: Configuration file 'missing.yml' not found.
#=> --exit 2
```
````
````

````
8
9  A successful run should output the version number:
10 ```bash
11 ./app --version
12 #=> v1.0.0
13 ```
````

B.5 `byexample`: Dynamic Output Matching

`byexample` uses wildcards (such as `<...>`) to perform pattern matching, preventing tests from failing due to legitimate output drift like changing process IDs or timestamps.

Example Markdown block tested by `byexample`:

````
1  When you start the worker, it will assign a dynamic Process ID and timestamp:
2
3  ```bash
4  $ ./worker start
5  Worker started with PID <...> at <...>
6  ```
7
8  Check the installed version (matches any minor/patch updates in the 2.x line):
9  ```bash
10 $ ./worker --version
11 v2.<...>
12 ```
````

C AI Coding Agent Instructions

The following documents define the instructions, workflow, and specialized skills used by the AI coding assistant throughout this project. These prompts specify the expected development process, coding conventions, testing requirements, and project-specific capabilities.

```
Before implementing a feature run the tests and after check the test again.
Add markdown tests for new feature
When writing code use go idiomatic code, pre allocate where you can and minimize
     GC pressure. Re-use code where possible.
DO NOT COMMIT IF NOT EXPLICITLY ASKED
```

Listing C.1: General AI coding instructions (AGENT.md)

```
---
name: grill-me
description: Interview the user relentlessly about a plan or design until
    reaching shared understanding, resolving each branch of the decision tree.
    Use when user wants to stress-test a plan, get grilled on their design, or
    mentions "grill me".
---

Interview me relentlessly about every aspect of this plan until we reach a
    shared understanding. Walk down each branch of the design tree, resolving
    dependencies between decisions one-by-one. For each question, provide your
    recommended answer and why.

Ask the questions one at a time.

If a question can be answered by exploring the codebase, explore the codebase
    instead.
```

Listing C.2: GrillMe skill definition (GRILLME.md)

```
---
name: makedo-agent-skill
description: Learn how to use makedo for markdown-based testing and embedding
license: MIT
metadata:
  audience: developers, coding agents
  workflow: testing, documentation
---

## What is makedo?

```

````
Makedo is a tool that brings integration testing directly inside your markdown documentation. Documentation becomes the tests, and tests become documentation.

Makedo allows you to:
- Verify that your documentation is still saying the truth
- Run integration testing on your cli tool using markdown
- Run all the codes in a markdown file (useful for setup)

**Note for the agent**: This skill file is already an example of `makedo` usage! The code blocks below are followed by HTML comments that `makedo` uses to validate the output. You can run `makedo test skills/SKILL.md` to verify this file.

## When to use it

Use this tool when you need to:
- Write documentation that needs to be automatically tested against regressions.
- Embed CLI integration tests directly within your markdown files.
- Ensure that the commands in your README or docs actually produce the output they claim to produce.
- Use dynamic types like `${{date}}` or `${{uuid}}` to verify outputs that change on every run.

## Main command (test, embed, help)

Makedo works by looking for fenced code blocks followed by an HTML comment in this format: `<!-- directive content -->`.

### Help

You can check the main help command to see what makedo can do:

```bash
makedo -h
```
<!-- out A markdown-based task runner -->

### Test

The `test` command runs code blocks with "out" directives and verifies their output matches the expected pattern.

```bash
makedo test -h
```
<!-- out directive <content> -->
````

````
50
51 ### Embed
52
53 The `embed` command runs fenced code blocks and embeds their output directly
       into the markdown file.
54
55 ```bash
56 makedo embed -h
57 ```
58 <!-- out Run fenced code blocks and embed output into the markdown file -->
````

Listing C.3: MakeDo skill definition (MAKEDO.md)

D Survey of Repository Documentation Validation

This appendix presents the raw findings of the 62-repository survey discussed in Section 2.1. For each repository, we examined whether its primary documentation (README or dedicated guides) contained executable shell blocks, whether those blocks were validated in CI, and what tooling (if any) was used.

Methodology Note: Our survey exclusively analysed the Continuous Integration (CI) configuration files (e.g., GitHub Actions workflows) of each repository to determine if Markdown files were automatically validated upon code changes. This means that if a project relies solely on local scripts or manual checks executed by developers on their own machines, it was classified as having no CI validation. Our focus was strictly on automated, enforced checks at the CI level.

Table D.1: Survey of documentation validation practices across 62 open-source repositories.

Repository	README Shell	README URLs	Guide Shell	CI Valid.	Tool*
act	No	Yes	No	No	markdownlint
anthropic-sdk-python	Yes	Yes	Yes	No	None
bat	Yes	Yes	Yes	No	None
btop	Yes	Yes	Yes	No	None
buf	Yes	Yes	Yes	No	None
caddy	Yes	Yes	No	No	None
certbot	No	No	No	No	None
cheat.sh	Yes	Yes	Yes	No	None
cli	Yes	Yes	Yes	No	None
curl	No	Yes	Yes	Yes	Custom Perl scripts
dagger	Yes	Yes	Yes	No	None
datadog-agent	No	Yes	Yes	Yes	markdown-link-check
django	No	No	No	Yes	sphinx.ext.doctest
earthly	Yes	Yes	Yes	Yes	None
express	No	No	No	No	None
eza	No	Yes	No	No	None
fastapi	Yes	Yes	Yes	Yes	pytest
fd	Yes	Yes	Yes	No	None
fzf	Yes	Yes	Yes	No	None
gin	Yes	Yes	Yes	No	None
goreleaser	No	Yes	No	No	None

Continued on next page

Table D.1 – continued from previous page

Repository	README Shell	README URLs	Guide Shell	CI Valid.	Tool*
grafana	No	Yes	Yes	Yes	mdl
httpie	Yes	Yes	Yes	Yes	mdl
hyperfine	Yes	Yes	Yes	No	None
istio	No	Yes	No	No	None
jaeger	Yes	Yes	Yes	No	None
just	Yes	Yes	Yes	No	None
kibana	No	Yes	Yes	Yes	vale
langchain	Yes	Yes	Yes	No	None
laravel	Yes	Yes	No	No	None
lazygit	Yes	Yes	Yes	No	None
llama.cpp	Yes	Yes	Yes	Yes	Custom python script
loki	Yes	Yes	Yes	No	mdl
lore	Yes	Yes	Yes	No	None
mkcert	Yes	Yes	Yes	No	None
mlflow	Yes	Yes	Yes	Yes	None
nest	No	No	No	No	None
netdata	No	Yes	Yes	No	None
next.js	No	No	No	No	None
nginx	Yes	Yes	Yes	No	None
ollama	Yes	Yes	Yes	No	None
opa	No	Yes	Yes	Yes	markdownlint
openai-python	Yes	Yes	Yes	No	None
opentelemetry-collector	Yes	Yes	Yes	Yes	markdownlint
prometheus	Yes	Yes	Yes	No	None
pytorch	Yes	Yes	Yes	Yes	sphinx.ext.doctest
rails	Yes	Yes	Yes	No	None
ray	No	No	Yes	No	None
ripgrep	Yes	Yes	Yes	No	None
runner	No	Yes	Yes	No	None
semantic-release	No	Yes	No	No	None
spring-boot	No	No	No	No	None
svelte	No	Yes	No	No	None
task	No	Yes	No	No	None
telegraf	No	Yes	Yes	Yes	markdownlint
tensorflow	Yes	Yes	No	No	None
transformers	Yes	Yes	Yes	Yes	doctest
trivy	Yes	Yes	Yes	No	None
vault	Yes	Yes	No	No	None
vector	No	Yes	Yes	Yes	markdownlint
woodpecker	No	Yes	Yes	No	None
zoxide	Yes	Yes	Yes	Yes	markdownlint

* **Tooling categories:** Tools observed in this survey generally fall into three categories:

  - **Markdown formatters (`markdownlint`, `mdl`):** Linters that statically check Markdown syntax (e.g., heading levels, list indentation).
  - **Link checkers (`markdown-link-check`):** Tools that parse the document for URLs and perform HTTP requests to detect broken links.
  - **Prose linters (`vale`):** Tools that check English grammar, tone, and style-guide compliance.

Crucially, none of these tools execute the shell commands embedded in the documentation.

E Test Porting Example: `testscript` to MakeDo

This appendix provides a concrete example of a Hugo integration test ported from its original `testscript` format to a MakeDo Markdown document, illustrating the differences discussed in Section 4.4.2.

Listing E.1 shows the original `testscript` for the `server__watch_hugo_stats` test. Listing E.2 shows the same test ported to MakeDo.

```
hugo server  --renderToMemory &

waitServer
stopServer
! stderr .

exists hugo_stats.json

-- hugo.toml --
title = "Hugo Server Test"
baseURL = "https://example.org/"
disableKinds = ["taxonomy", "term", "sitemap"]
[module]
[[module.mounts]]
source = "hugo_stats.json"
target = "assets/watching/hugo_stats.json"
-- layouts/home.html --
<body>Home</body>
```

Listing E.1: The original `testscript` for `server__watch_hugo_stats`.

As seen in these listings, the MakeDo version is significantly more verbose. This verbosity arises primarily for two reasons. First, MakeDo requires explicit file creation using shell commands (e.g., `cat << 'EOF' > ...`), whereas `testscript` provides a compact, built-in archive format for declaring file contents. Second, MakeDo requires every executed command to be wrapped within a fenced code block and accompanied by explicit Markdown directives for assertions, adding structural overhead compared to the terse, line-oriented syntax of `testscript`.

However, there is also a notable difference in how both frameworks handle long-running background processes. In the `testscript` example, waiting for the server to start (`waitServer`) and stopping it (`stopServer`) require custom framework helper functions written in a separate Go file. By contrast, MakeDo has built-in support for long-running processes launched in the background (using `&`). A simple `curl` command, used as a readiness probe in an explicit directive (`<!-- cmd curl -s http://localhost:8995 -->`), is sufficient to check when the server is ready before moving on to the next code block.

````
# Test server__watch_hugo_stats

### Setup `hugo.toml`

```bash
cat << 'EOF' > hugo.toml
title = "Hugo Server Test"
baseURL = "https://example.org/"
disableKinds = ["taxonomy", "term", "sitemap"]
[module]
[[module.mounts]]
source = "hugo_stats.json"
target = "assets/watching/hugo_stats.json"
EOF
```
<!-- checkpath hugo.toml -->

### Setup `layouts/home.html`

```bash
mkdir -p layouts
cat << 'EOF' > layouts/home.html
<body>Home</body>
EOF
```
<!-- checkpath layouts/home.html -->

### Start Server

```bash
hugo server --renderToMemory -p 8995 2> hugo_errors.log &
```
<!-- cmd curl -s http://localhost:8995 -->

### Verify stats file

```bash
test -e hugo_stats.json
```
<!-- exit 0 -->

### Check errors

```bash
cat hugo_errors.log
```
<!-- !outr . -->

### Cleanup

```bash
rm -rf hugo.toml layouts hugo_stats.json hugo_errors.log
```
````

Listing E.2: The ported MakeDo Markdown test for `server__watch_hugo_stats`.

Technical
University of
Denmark

Building 324
2800 Kgs. Lyngby
Tlf. 4525 1700

https://www.compute.dtu.dk